

2026학년도

고교단계 일학습병행
산학일체형 도제학교

외부평가 학습교재
[SW개발_L3]



세명컴퓨터고등학교

목 차

I. 필수 능력단위	3
1. 애플리케이션 배포	3
2. 애플리케이션 테스트 수행	15
3. 응용SW 기초 기술 활용	22
4. 개발자 환경 구축	29
5. UI 테스트	35
6. 프로그래밍 언어 활용	42
7. 화면 구현	49
8. SQL 활용	55
9. 프로그래밍 언어 응용	62
II. 실전 모의고사	69
1. 1회	69
2. 2회	72
3. 3회	75
4. 4회	78
5. 정답 및 해설	81

I. 필수 능력단위

1. 애플리케이션 배포 [LM2001020214_23v6]

학습 1 애플리케이션 배포 환경 구성하기

1-1. 애플리케이션 배포 환경 구성

학습목표 배포 대상 시스템의 특징을 파악하고, 배포에 필요한 도구와 절차를 구성할 수 있다.

가. 배포 환경의 개념

- 1) 배포 환경이란 개발이 완료된 소스코드를 실행 가능한 형태로 변환(빌드)하고, 이를 운영 서버까지 전달하는 데 필요한 전체 시스템·도구·절차의 집합을 말한다.
- 2) 배포 환경에는 형상관리 도구, 빌드 도구, 정적/동적 테스트 도구, 배포 자동화 도구가 포함되며, 각 도구는 서로 연계되어 하나의 파이프라인을 구성한다.

나. 언어 유형별 빌드 방식

- 1) 컴파일 언어(C, C++ 등)는 소스 전체를 기계어로 변환한 실행 파일을 만들며 실행 속도는 빠르지만 플랫폼마다 다시 빌드해야 한다.
- 2) 바이트코드 언어(Java, C# 등)는 중간 코드로 컴파일한 후 가상머신(JVM, CLR)에서 실행되어 이식성이 높다.
- 3) 인터프리터 언어(Python, JavaScript 등)는 소스를 한 줄씩 해석·실행하므로 별도의 빌드 없이 즉시 실행할 수 있어 개발·검증 주기가 짧다.

다. 배포 대상과 웹 기반 배포 환경

- 1) 배포 대상에는 실행 파일, 라이브러리, 환경설정 파일, 인터페이스 정의서 등이 포함되며 유형별로 배포 절차가 달라질 수 있다.
- 2) 웹 서버는 HTML·이미지 등 정적 자원을 처리하고, WAS(Web Application Server)는 비즈니스 로직이 담긴 동적 프로그램을 실행한다. 두 서버는 통상 함께 구성되어 하나의 서비스 환경을 이룬다.

[표 1-1] 웹 서버와 WAS 비교

구분	역할	대표 예시
웹 서버	정적 콘텐츠 응답, 부하 분산	Apache HTTP Server, Nginx
WAS	동적 로직 처리, DB 연동	Tomcat, JBoss, WebLogic

라. 배포 파이프라인의 이해

- 1) 배포 파이프라인은 소스 커밋부터 운영 반영까지의 전체 과정을 자동화한 흐름으로, 일반적으로 형상관리 → 빌드 → 정적분석 → 테스트 → 패키징 → 배포의 단계를 거친다.
- 2) 각 단계는 이전 단계의 결과가 성공했을 때만 다음 단계로 진행되도록 구성하여, 결함이 있는 코드가 운영 환경까지 전달되는 것을 방지한다.

마. 온프레미스와 클라우드 배포 환경

- 1) 온프레미스 환경은 자체 데이터센터에 서버를 직접 구축·운영하는 방식이며, 클라우드 환경은 AWS·Azure 등 클라우드 사업자의 인프라를 임대하여 사용하는 방식이다.
- 2) 클라우드 환경에서는 컨테이너(Docker)와 오케스트레이션 도구(Kubernetes)를 활용해 배포 단위를 표준화하고 확장성을 높이는 경우가 많다.

▶ 실무 Tip

배포 환경을 새로 구축할 때는 개발-테스트-운영 3단계 환경을 최소한으로 분리하는 것이 일반적이다.

환경별로 접속 계정과 데이터베이스를 분리하여, 테스트 중 실수로 운영 데이터가 훼손되는 사고를 예방한다.

1-2. 애플리케이션 배포 환경 설정

학습목표 *형상관리 체계와 배포 도구를 설정하여 반복 가능한 배포 절차를 수립할 수 있다.*

가. 배포 단위와 패키징

1) 애플리케이션은 단일 파일이 아닌 패키지 단위로 배포되며, Java 기반 시스템에서는 jar·war·ear 형태가 널리 쓰인다.

나. 형상관리 시스템

1) 형상관리는 소스의 변경 이력을 체계적으로 관리하는 활동으로, 형상 항목·기준선·저장소(Repository) 개념을 바탕으로 한다.

2) 체크아웃(Check-Out)은 저장소에서 소스를 가져오는 작업이고, 체크인(Check-In)은 수정이 끝난 소스를 저장소에 반영하는 작업이다. 두 작업을 통해 동시 수정으로 인한 충돌을 예방할 수 있다.

다. 배포 유형과 역할 분담

1) 배포는 장애 시 즉시 반영하는 긴급 배포, 검증 절차를 거치는 일반 배포, 주기적으로 전체를 통합하는 정기(통합) 배포로 구분한다.

2) 배포 관리자는 도구를 운영하고, PM/PL은 결과를 검토·승인하며, 개발자는 소스 수정과 단위 테스트를 담당한다.

라. 배포 도구의 설치와 연동

1) 서버 측에는 빌드·배포 자동화 도구(Jenkins, GitLab CI 등)를 설치하고, 개발자 PC에는 형상관리 클라이언트와 IDE 플러그인을 연동하여 하나의 작업 흐름으로 통합한다.

2) 도구 간 연동 시 접근 계정과 권한을 최소한으로 부여하는 최소 권한 원칙을 적용하여 보안 사고 가능성을 낮춘다.

[표 1-1a] 배포 유형별 특징

배포 유형	적용 시점	특징
긴급 배포(Hotfix)	장애 발생 시 즉시	최소한의 검증 후 신속 반영
일반 배포	정기 일정에 따라	충분한 테스트를 거쳐 반영
정기(통합) 배포	주기적(예: 스프린트 단위)	여러 변경사항을 묶어 한 번에 반영

학습 2 애플리케이션 소스 검증하기

2-1. 애플리케이션 소스 확인

학습목표 *형상관리 서버로부터 소스를 반출하여 검증 대상을 식별할 수 있다.*

가. 소스 반출의 목적

- 1) 형상관리 서버에 보관된 최신 소스를 반출(Check-Out)하여 수정·검증 작업 환경을 준비한다.

나. 코드 인스펙션

- 1) 코드 인스펙션은 프로그램을 실행하지 않고 소스코드 자체를 분석하는 정적 검증 기법으로, 코딩 규칙 위반·성능 저하 요인·잠재적 결함을 사전에 찾아낸다.
- 2) 점검 규칙(Rule)은 심각도에 따라 필수 수정 항목과 권고 항목으로 구분되며, 정규표현식을 이용해 특정 코드 패턴을 탐지하기도 한다.

다. 소스 변경 이력 관리

- 1) 반출(Check-Out)과 반입(Check-In) 과정에서 발생하는 변경 사항은 커밋 메시지와 함께 기록되어, 이후 문제 발생 시 원인이 된 변경 시점을 추적하는 데 활용된다.
- 2) 브랜치 전략(예: Git-flow)을 적용하면 기능 개발, 배포 준비, 긴급 수정 작업을 서로 분리하여 동시에 진행할 수 있다.

▶ 예시: 코드 인스펙션 점검 항목

- 사용하지 않는 변수·주석 처리된 코드가 남아 있는가
- 예외 처리 없이 외부 자원(파일, 네트워크)에 접근하는 코드가 있는가
- 하드코딩된 비밀번호·URL 등 민감정보가 포함되어 있는가

2-2. 애플리케이션 소스 검증

학습목표 정적·동적 검증 도구를 활용하여 소스코드의 품질과 안정성을 확인할 수 있다.

가. 정적 검증과 동적 검증

- 1) 정적 테스트 도구는 프로그램을 실행하지 않고 소스 구조·규칙 위반을 분석하여 보안 취약점과 코드 품질 문제를 사전에 제거한다.
- 2) 동적 테스트 도구는 실제 실행 환경에서 프로그램을 구동하며 기능·성능상의 오류를 찾아내어 운영 안정성을 확보한다.

나. 테스트 프레임워크 활용

- 1) JUnit(Java), pytest(Python) 등 테스트 프레임워크를 이용하면 테스트 코드를 재사용하여 반복 검증을 자동화할 수 있다.
- 2) 검증 결과에서 실제 결함과 오탐(False Positive)을 구분한 뒤 개발 조직에 공유하여 개선을 유도한다.

[표 1-2] 정적 테스트와 동적 테스트

구분	실행 여부	주요 목적	대표 도구
정적 테스트	미실행	코딩 규칙·보안 취약점 사전 발견	SonarQube, PMD
동적 테스트	실행	실제 동작·성능 결함 발견	JUnit, Selenium

다. 검증 결과 관리

- 1) 검증 도구가 산출한 결함 목록은 심각도별로 분류하여 담당자에게 배정하고, 처리 현황을 대시보드로 관리하면 진행 상황을 한눈에 파악할 수 있다.
- 2) 동일한 패턴의 결함이 반복적으로 발견되면, 코딩 표준이나 팀 내 개발 가이드에 해당 규칙을 추가하여 재발을 방지한다.

학습 3 애플리케이션 빌드하기

3-1. 애플리케이션 빌드 실행

학습목표 지속적 통합 환경에서 빌드 스크립트를 실행하여 실행 파일을 생성할 수 있다.

가. 지속적 통합(CI)

1) 지속적 통합(Continuous Integration)은 개발자가 변경한 코드를 자주 통합·빌드·검증하여 통합 과정에서 발생하는 오류를 조기에 발견하는 개발 방식이다.

나. 빌드 단위와 버전 관리

1) 빌드는 화면, 온라인, 배치, 데이터 영역 등으로 구분하여 관리하며, 변경 유형(주 버전·부 버전·패치)에 따라 버전 번호를 체계적으로 부여한다.

다. 빌드 스크립트

1) 빌드 스크립트는 컴파일, 테스트, 패키징, 코드 인스펙션 연계 과정을 자동화한 파일로, Maven의 pom.xml, Gradle의 build.gradle 등이 대표적이다.

라. 빌드 자동화 서버

1) Jenkins, GitLab CI/CD와 같은 빌드 자동화 서버는 소스 변경을 감지하여 빌드·테스트를 자동 실행하고, 결과를 팀에 즉시 통보한다.

2) 빌드 파이프라인을 단계별(Stage)로 구성하면 각 단계의 성공·실패 여부를 시각적으로 확인할 수 있어 문제 구간을 빠르게 특정할 수 있다.

▶ 예시: 빌드 스크립트 실행 순서

- ① 소스 체크아웃 → ② 의존성 라이브러리 다운로드 → ③ 컴파일 → ④ 단위 테스트 실행
- ⑤ 코드 인스펙션 → ⑥ 패키징(jar/war 생성) → ⑦ 산출물 저장소 업로드

3-2. 애플리케이션 실행 검증

학습목표 빌드 결과와 테스트 커버리지를 분석하여 오류를 진단하고 대응할 수 있다.

가. 빌드 결과 확인

- 1) 빌드 로그를 통해 컴파일 성공 여부, 경고·오류 메시지를 확인한다.

나. 테스트 커버리지 분석

- 1) 커버리지는 테스트가 소스코드를 얼마나 실행했는지 나타내는 지표로, 구문(Statement)·분기(Branch) 커버리지가 널리 쓰인다.

다. 오류 대응

- 1) 오류 원인을 로그와 스택 트레이스로 분석하여 담당 개발자에게 전달하고 재빌드 후 재검증한다.

학습 4 애플리케이션 배포하기

4-1. 애플리케이션 배포

학습목표 운영 환경의 특성을 고려하여 절차에 따라 애플리케이션을 배포할 수 있다.

가. 운영 환경의 특성

- 1) 운영 환경은 보안, 네트워크 접근 통제, 계정 관리 측면에서 개발·테스트 환경보다 엄격한 제약을 받는다.

나. 배포 절차

- 1) 배포 전 운영 서버의 IP, 접속 계정, 디렉터리 구조를 확인하고 이행의뢰서(배포 요청서)를 작성한다.
- 2) 배포 시각은 서비스 영향이 적은 시간대(야간·점검 시간)를 선정하는 것이 일반적이다.

4-2. 애플리케이션 운영 및 복원

학습목표 배포 후 정상 동작을 확인하고 장애 발생 시 신속히 복원할 수 있다.

가. 배포 후 점검

- 1) 서비스 주요 기능을 점검하여 배포가 정상적으로 반영되었는지 확인한다.

나. 롤백(Rollback)

- 1) 장애 발생 시 이전 정상 버전으로 되돌리는 롤백 절차를 사전에 준비해 두어야 서비스 중단 시간을 최소화할 수 있다.

다. 지속적 운영 관리

- 1) 배포 이후에도 모니터링 체계를 통해 자원 사용률·오류율을 지속적으로 관찰하여 안정적인 서비스를 유지한다.

애플리케이션 배포 - 학습내용 확인하기

1. 배포 환경과 웹 기반 배포

[핵심 요약]

- 가. 배포 환경은 애플리케이션을 빌드·배포하기 위한 서버, 도구, 시스템의 총체이다.
- 나. 언어 특성에 따라 컴파일·바이트코드·인터프리터 방식으로 빌드된다.
- 다. 웹 서버는 정적 자원을, WAS는 동적 로직을 처리한다.

[빈칸 채우기]

- ① 소스코드를 실행 가능한 형태로 변환하는 과정을 ()라고 한다.
- ② 정적 자원을 처리하는 서버는 ()이고, 동적 로직을 처리하는 서버는 ()이다.

2. 배포 환경 설정과 형상관리

[핵심 요약]

- 가. 애플리케이션은 패키지 단위(jar, war, ear)로 배포된다.
- 나. 형상관리는 체크아웃과 체크인을 통해 변경 이력을 관리한다.

[빈칸 채우기]

- ① 저장소에서 소스를 가져오는 작업을 ()이라 한다.
- ② 수정한 소스를 다시 저장소에 반영하는 작업을 ()이라 한다.

3. 소스 확인과 코드 인스펙션

[핵심 요약]

- 가. 소스 확인 단계는 형상관리 서버에서 소스를 반출하는 것으로 시작한다.
- 나. 코드 인스펙션은 실행 없이 소스를 분석하는 정적 검증 기법이다.

[빈칸 채우기]

- ① 실행하지 않고 소스를 분석하는 검증 방법을 ()이라고 한다.
- ② 코드 인스펙션은 () 테스트 기법에 해당한다.

4. 소스 검증 도구

[핵심 요약]

- 가. 정적 테스트 도구는 실행 전, 동적 테스트 도구는 실행 중 오류를 검증한다.
- 나. 테스트 프레임워크를 사용하면 반복 검증을 자동화할 수 있다.

[빈칸 채우기]

- ① 실행 없이 코드를 분석하는 도구는 () 테스트 도구이다.
- ② Java 진영의 대표적인 테스트 프레임워크는 ()이다.

5. 빌드와 지속적 통합

[핵심 요약]

- 가. 빌드는 소스코드를 컴파일·패키징하는 과정이다.
- 나. CI는 빌드와 테스트를 자동으로 반복 수행하는 환경이다.

[빈칸 채우기]

- ① 빌드·테스트를 자동으로 반복하는 환경을 ()라고 한다.
- ② 빌드 절차를 자동화한 파일을 ()라고 한다.

6. 배포 절차와 운영

[핵심 요약]

- 가. 배포 전 운영 서버 정보와 접근 권한을 반드시 확인해야 한다.
- 나. 장애 발생 시 이전 버전으로 되돌리는 것을 롤백이라 한다.

[빈칸 채우기]

- ① 배포 후 문제가 발생하면 이전 버전으로 ()할 수 있어야 한다.
- ② 운영 환경에서는 () 정책과 접근 권한 관리가 중요하다.

[정리 문제 - 서술형]

1. 애플리케이션 배포 절차를 4단계로 순서대로 서술하시오.
2. 코드 인스펙션과 동적 테스트의 차이점을 비교하여 설명하시오.
3. 배포 후 장애가 발생했을 때 취해야 할 조치를 서술하시오.

[정답]

1. 배포 환경 구성 → 소스 검증 → 빌드 → 운영 배포 순으로 진행한다.
2. 코드 인스펙션은 실행하지 않고 소스를 분석하는 정적 기법이며, 동적 테스트는 프로그램을 실제로 실행하여 동작을 검증하는 기법이다.
3. 장애 원인을 신속히 파악하고, 서비스 영향을 최소화하며, 필요 시 이전 버전으로 롤백한 뒤 원인을 수정하여 재배포한다.

애플리케이션 배포 - 형성평가

1. 애플리케이션 배포의 목적으로 가장 적절한 것은?
① 소스코드를 새로 작성하기 위함 ② 운영 환경에서 실행 가능하도록 전달하기 위함
③ 테스트 케이스를 설계하기 위함 ④ 데이터베이스 구조를 정의하기 위함
2. 다음 중 정적 테스트에 해당하는 것은?
① 프로그램 실제 실행 ② 코드 인스펙션 ③ 사용자 인수 테스트 ④ 부하 테스트
3. 웹 애플리케이션에서 동적 비즈니스 로직을 처리하는 서버는?
① 웹 서버 ② DB 서버 ③ WAS ④ 파일 서버
4. 형상관리에서 수정한 소스를 저장소에 반영하는 작업은?
① 체크아웃 ② 체크인 ③ 빌드 ④ 배포
5. 지속적 통합(CI)의 주요 목적으로 옳은 것은?
① 서버 보안 강화 ② 빌드와 테스트의 자동화 ③ 데이터 암호화 ④ 네트워크 속도 개선
6. Java 웹 애플리케이션 배포에 주로 사용되는 패키지 형식은?
① exe ② war ③ zip ④ txt
7. 배포 환경 구성 요소로 가장 거리가 먼 것은?
① 형상관리 도구 ② 빌드 도구 ③ 테스트 도구 ④ 워드프로세서
8. 다음 중 동적 테스트의 특징으로 옳은 것은?
① 소스를 실행하지 않는다 ② 코드 규칙만 점검한다 ③ 실행 중 동작을 검증한다
④ 형상관리만 수행한다
9. 배포 절차의 올바른 순서는?
① 빌드→배포→검증→구성 ② 환경 구성→소스 검증→빌드→배포 ③ 검증→구성→배포→빌드
④ 배포→빌드→검증→구성
10. 운영 환경의 특징으로 옳지 않은 것은?
① 보안이 중요하다 ② 접근 권한 관리가 필요하다 ③ 자유로운 테스트가 가능하다
④ 서비스 안정성이 중요하다
11. 소스코드를 실행 가능한 형태로 변환하는 과정을 무엇이라 하는가?
12. 배포 후 문제가 발생했을 때 이전 버전으로 되돌리는 작업을 무엇이라 하는가?
13. 실행하지 않고 소스를 분석하는 정적 검증 기법의 이름을 쓰시오.
14. 저장소에서 소스를 가져오는 형상관리 작업의 명칭을 쓰시오.
15. 웹 서버와 WAS의 차이를 한 문장으로 서술하시오.

[정답 및 해설]

문항	정답	해설
1	②	배포는 운영 환경에서 실행 가능하도록 전달하는 과정이다.
2	②	코드 인스펙션은 실행 없이 소스를 분석하는 정적 기법이다.
3	③	WAS는 동적 비즈니스 로직을 처리한다.
4	②	체크인은 수정 소스를 저장소에 반영하는 작업이다.
5	②	CI는 빌드·테스트 자동화를 목적으로 한다.
6	②	Java 웹 애플리케이션은 war로 배포되는 경우가 많다.
7	④	워드프로세서는 배포 환경 구성 요소가 아니다.
8	③	동적 테스트는 실행 중 동작을 검증한다.
9	②	환경 구성→검증→빌드→배포 순이다.
10	③	운영 환경은 자유로운 테스트가 제한된다.
11	빌드(Build)	소스를 실행 가능한 형태로 변환하는 과정이다.
12	롤백(Rollback)	이전 정상 버전으로 되돌리는 작업이다.
13	코드 인스펙션	실행 없이 코드를 분석하는 정적 기법이다.
14	체크아웃(Check-Out)	저장소에서 소스를 가져오는 작업이다.
15	-	웹 서버는 정적 자원을, WAS는 동적 로직을 처리한다.

2. 애플리케이션 테스트 수행 [LM2001020227_23v6]

학습 1 애플리케이션 테스트 수행하기

1-1. 애플리케이션 테스트 수행

학습목표 테스트 계획에 따라 서버·화면 모듈, 데이터 입출력, 인터페이스가 요구사항을 충족하는지 테스트할 수 있다.

가. 소프트웨어 개발 생명주기(SDLC)와 V-모델

- 1) V-모델은 요구사항·분석·설계·구현 단계와 이에 대응하는 인수·시스템·통합·단위 테스트 단계를 좌우로 배치하여, 개발 산출물과 테스트 산출물의 대응 관계를 명확히 보여준다.
- 2) 테스트는 일반적으로 단위 테스트 → 통합 테스트 → 시스템 테스트 → 인수 테스트 순서로 진행된다.

나. 테스트 유형별 특징

- 1) 단위 테스트는 모듈·컴포넌트 단위에서 결함을 찾는 테스트로, 개발자가 직접 수행하며 구조 기반(화이트박스)과 명세 기반(블랙박스) 방법으로 나뉜다.
- 2) 통합 테스트는 모듈 간 인터페이스와 상호작용을 검증하며, 빅뱅·상향식·하향식·샌드위치 방식 등이 있다.
- 3) 시스템 테스트는 통합된 전체 시스템이 요구사항명세서·유스케이스에 명시된 기능·성능을 만족하는지 확인한다.
- 4) 인수 테스트는 사용자·이해관계자가 실제 운영 여부를 결정하기 위해 수행하며, 사용자 인수·운영상 인수·계약 인수·규정 인수·알파·베타 테스트 등으로 구분된다.

[표 2-1] 단위 테스트 방법 비교

테스트 방법	테스트 관점	테스트 목적
구조 기반(화이트박스)	프로그램 내부 구조·복잡도 검증	제어 흐름, 조건·결정 커버리지 확인
명세 기반(블랙박스)	요구사항·명세서 기반 실행	동등 분할, 경계값 분석

다. 테스트 케이스 설계

- 1) 테스트 케이스는 입력값, 실행 조건, 예상 결과로 구성되며, 요구사항명세서의 각 기능 항목에 대응하도록 작성해야 누락 없이 검증할 수 있다.
- 2) 정상 입력값뿐 아니라 경계값·예외 입력값(빈 값, 최댓값, 잘못된 형식 등)을 포함해야 결함 발견율이 높아진다.

라. 테스트 자동화 도구

- 1) 테스트 관리 도구는 테스트 케이스 작성과 결함 추적·기록 기능을 제공하며 (ClearQuest, DevTrack 등), 테스트 실행 도구는 반복 테스트를 자동으로 수행한다 (JUnit, Selenium 등).
- 2) 테스트 자동화는 반복적인 회귀 테스트에 특히 효과적이며, CI 파이프라인에 통합하면 코드 변경 시마다 자동으로 검증이 이루어진다.

▶ 예시: 테스트 케이스 작성 항목

- 테스트 ID / 테스트 대상 기능 / 사전 조건
- 입력값 및 실행 절차 / 기대 결과 / 실제 결과 / 통과 여부(Pass·Fail)

1-2. 애플리케이션 결함 관리

학습목표 테스트로 발견한 결함을 유형별로 기록하고 원인을 분석하여 개선 방안을 도출할 수 있다.

가. 결함의 정의

- 1) 결함(Defect)이란 프로그램과 명세서 간의 차이, 또는 기대 결과와 실제 관찰 결과 간의 차이를 말한다.

나. 결함 관리 프로세스

- 1) 결함 관리는 계획 → 기록 → 검토 → 수정 → 재확인 → 상태 추적 → 최종 분석·보고서 작성의 7단계로 진행된다.
- 2) 결함 상태는 등록(Open) → 할당(Assigned) → 수정(Fixed) → 재테스트(Retest) → 종료(Closed) 등으로 전이되며, 재현되지 않으면 반려(Reject)로 처리한다.

[표 2-2] 결함 관리 프로세스 주요 단계

단계	주요 활동
결함 기록	테스터가 발견한 결함 정보를 결함 관리 DB에 등록
결함 검토	주요 내용을 검토하고 담당 개발자에게 배정
결함 수정	개발자가 원인을 분석하여 프로그램을 수정
결함 재확인	수정 완료 후 테스터가 재테스트로 조치 결과 확인

다. 결함 기록 항목

- 1) 결함을 기록할 때는 재현 절차, 발생 환경(OS·브라우저·버전), 심각도, 첨부 자료(스크린샷·로그)를 함께 남겨야 담당 개발자가 원인을 빠르게 파악할 수 있다.
- 2) 모호한 결함 설명(예: '동작이 이상함')은 재현이 어려워 처리가 지연되므로, 관찰된 현상과 기대 동작을 구체적으로 구분하여 작성한다.

학습 2 애플리케이션 결함 조치하기

2-1. 애플리케이션 결함 조치 우선순위 결정

학습목표 결함의 심각도와 영향 범위를 분석하여 조치 우선순위를 결정할 수 있다.

가. 심각도(Severity)와 우선순위(Priority)

- 1) 심각도는 결함이 시스템에 미치는 영향의 크기를, 우선순위는 결함을 얼마나 시급히 처리해야 하는지를 나타내며 두 기준은 독립적으로 판단한다.
- 2) 예를 들어 로그인 실패처럼 심각도는 높지만 발생 빈도가 낮은 결함과, 화면 오탈자처럼 심각도는 낮지만 사용자 노출이 잦은 결함은 우선순위 판단이 달라질 수 있다.

나. 우선순위 결정 기준

- 1) 비즈니스 영향도, 재현 빈도, 대체 수단(우회 방법) 존재 여부, 출시 일정을 종합적으로 고려하여 결정한다.

[표 2-3] 결함 우선순위 등급 기준(예시)

등급	기준	처리 목표 시간
긴급(Critical)	서비스 전체 중단, 대체 수단 없음	즉시(당일)
높음(High)	핵심 기능 장애, 대체 수단 제한적	1~2일 이내
보통(Medium)	일부 기능 불편, 우회 방법 존재	정기 배포 시 반영
낮음(Low)	화면 표시 오류 등 경미한 문제	여유 있을 때 처리

2-2. 애플리케이션 결함 조치 관리

학습목표 결함 조치 결과를 관리하고 재발 방지 대책을 수립할 수 있다.

가. 결함 조치 관리

- 1) 결함 조치가 완료되면 회귀 테스트(Regression Test)를 수행하여 수정 사항이 다른 기능에 영향을 주지 않는지 확인한다.

나. 종료 기준(Exit Criteria)

- 1) 중대한 결함이 모두 조치되고, 남은 결함의 영향이 제한적이며 예산·일정상 허용 가능한 수준일 때 테스트를 종료할 수 있다.

다. 결함 분석 및 보고

- 1) 결함 유형·발생 원인을 통계적으로 분석하여 향후 개발 프로세스 개선에 반영하고, 최종 테스트 결과 보고서를 작성한다.
- 2) 결함 밀도(코드 규모 대비 결함 수), 결함 제거율과 같은 지표를 활용하면 품질 수준을 정량적으로 파악할 수 있다.

애플리케이션 테스트 수행 - 학습내용 확인하기

1. V-모델과 테스트 유형

[핵심 요약]

- 가. V-모델은 개발 단계와 테스트 단계의 대응 관계를 보여준다.
- 나. 테스트는 단위→통합→시스템→인수 순으로 진행된다.

[빈칸 채우기]

- ① 모듈 단위로 결함을 찾는 가장 작은 단위의 테스트를 () 테스트라 한다.
- ② 최종 사용자가 운영 여부를 결정하기 위해 수행하는 테스트를 () 테스트라 한다.

2. 구조 기반과 명세 기반

[핵심 요약]

- 가. 구조 기반 테스트는 화이트박스, 명세 기반 테스트는 블랙박스 테스트라 한다.
- 나. 통합 테스트는 빅뱅, 상향식, 하향식 등의 방식으로 진행된다.

[빈칸 채우기]

- ① 프로그램 내부 구조를 검증하는 테스트를 () 테스트라 한다.
- ② 명세서 기반으로 입력·출력을 확인하는 기법을 () 분석이라 한다.

3. 결함의 정의와 관리 프로세스

[핵심 요약]

- 가. 결함은 명세서와 실제 결과의 차이이다.
- 나. 결함 관리는 계획-기록-검토-수정-재확인-추적-보고의 절차로 진행된다.

[빈칸 채우기]

- ① 발견된 결함 정보를 등록하는 단계를 결함 ()이라 한다.
- ② 수정 완료 후 다시 테스트하여 확인하는 단계를 결함 ()이라 한다.

4. 우선순위와 종료 기준

[핵심 요약]

- 가. 심각도와 우선순위는 서로 다른 기준으로 판단한다.
- 나. 결함 조치 후에는 회귀 테스트로 부작용 여부를 확인한다.

[빈칸 채우기]

- ① 결함이 시스템에 미치는 영향의 크기를 ()라 한다.
- ② 수정 사항이 다른 기능에 영향을 주지 않는지 확인하는 테스트를 () 테스트라 한다.

[정리 문제 - 서술형]

1. 애플리케이션 테스트의 4단계 유형을 순서대로 나열하고 각각을 한 문장으로 설명하시오.
2. 심각도와 우선순위의 차이를 예를 들어 설명하시오.
3. 결함 관리 프로세스 7단계를 서술하시오.
4. 테스트 케이스에 반드시 포함되어야 할 구성 요소를 세 가지 이상 서술하시오.
5. 결함을 기록할 때 재현 절차를 구체적으로 남겨야 하는 이유를 서술하시오.

[정답]

1. 단위 테스트(모듈 단위 검증) → 통합 테스트(모듈 간 연동 검증) → 시스템 테스트(전체 기능·성능 검증) → 인수 테스트(사용자의 운영 여부 결정).
2. 심각도는 결함이 시스템에 미치는 영향 크기이고 우선순위는 처리의 시급성이다. 예를 들어 로그인 불가 결함은 심각도·우선순위 모두 높지만, 자주 쓰지 않는 화면의 오타자는 심각도는 낮고 우선순위도 낮게 책정될 수 있다.
3. 결함 관리 계획 → 결함 기록 → 결함 검토 → 결함 수정 → 결함 재확인 → 상태 추적·모니터링 → 최종 결함 분석 및 보고서 작성.
4. 입력값, 실행 조건(사전 조건), 실행 절차, 기대 결과, 실제 결과와 통과 여부가 포함되어야 한다.
5. 재현 절차가 모호하면 개발자가 결함을 재현하지 못해 원인 파악과 수정이 지연되므로, 발생 환경과 관찰된 현상을 구체적으로 남겨야 신속한 조치가 가능하다.

애플리케이션 테스트 수행 - 형성평가

1. V-모델에서 요구사항 분석 단계에 대응하는 테스트는?
① 단위 테스트 ② 통합 테스트 ③ 시스템 테스트 ④ 인수 테스트
2. 개발자가 직접 수행하며 가장 작은 단위를 검증하는 테스트는?
① 단위 테스트 ② 통합 테스트 ③ 인수 테스트 ④ 회귀 테스트
3. 프로그램 내부 구조를 검증하는 화이트박스 테스트 기반은?
① 명세 기반 ② 구조 기반 ③ 사용자 기반 ④ 성능 기반
4. 모듈 간 인터페이스 연동을 검증하는 테스트는?
① 단위 테스트 ② 통합 테스트 ③ 인수 테스트 ④ 코드 인스펙션
5. 결함의 정의로 가장 적절한 것은?
① 새로운 기능 요구 ② 명세서와 실제 결과의 차이 ③ 테스트 케이스 개수 ④ 소스코드 라인 수
6. 결함 관리 프로세스에서 가장 먼저 수행하는 단계는?
① 결함 수정 ② 결함 재확인 ③ 결함 관리 계획 ④ 결함 상태 추적
7. 수정 사항이 다른 기능에 영향을 주지 않는지 확인하는 테스트는?

- ① 회귀 테스트 ② 인수 테스트 ③ 단위 테스트 ④ 통합 테스트

8. 결함의 시급한 처리 정도를 나타내는 개념은?

- ① 심각도 ② 우선순위 ③ 커버리지 ④ 재현율

9. 사용자가 실제 업무 환경에서 시스템 인수 여부를 확인하는 테스트는?

- ① 단위 테스트 ② 통합 테스트 ③ 인수 테스트 ④ 코드 인스펙션

10. 테스트를 종료할 수 있는 조건으로 가장 적절한 것은?

- ① 모든 결함이 존재해도 무관 ② 중대 결함이 조치되고 잔여 결함 영향이 제한적 ③ 일정이 지나면 무조건 종료 ④ 예산과 무관하게 지속

11. 명세서 기반으로 입력값의 경계를 검증하는 기법의 명칭을 쓰시오.

12. 결함 등록부터 종료까지의 상태 흐름을 관리하는 것을 무엇이라 하는가?

13. 통합 테스트에서 하위 모듈부터 결합하는 방식의 명칭을 쓰시오.

14. 코드 규모 대비 결함 수를 나타내는 품질 지표를 쓰시오.

15. 결함 재현이 안 될 때 부여하는 결함 상태의 명칭을 쓰시오.

[정답 및 해설]

문항	정답	해설
1	④	요구사항 분석은 인수 테스트와 대응된다.
2	①	단위 테스트는 개발자가 직접 수행한다.
3	②	화이트박스는 구조 기반 테스트이다.
4	②	통합 테스트는 모듈 간 인터페이스를 검증한다.
5	②	결함은 명세서와 실제 결과의 차이이다.
6	③	결함 관리는 계획 수립부터 시작한다.
7	①	회귀 테스트는 부작용 여부를 확인한다.
8	②	우선순위는 처리의 시급성을 의미한다.
9	③	인수 테스트는 실사용 환경에서 수행한다.
10	②	중대 결함 조치와 잔여 결함의 제한적 영향이 종료 조건이다.
11	경계값 분석	입력값의 경계 부분을 집중 검증하는 기법이다.
12	결함 상태 추적(모니터링)	결함의 생애주기를 관리하는 활동이다.
13	상향식(Bottom-Up)	하위 모듈부터 결합하여 통합하는 방식이다.
14	결함 밀도	코드 규모 대비 결함 수를 나타내는 지표이다.
15	반려(Reject)	재현되지 않는 결함에 부여하는 상태이다.

3. 응용SW 기초 기술 활용 [LM2001020232_23v5]

학습 1 네트워크 기초 활용하기

1-1. 네트워크 프로토콜 활용

학습목표 네트워크 계층 구조의 역할을 구별하고 응용소프트웨어 특성에 맞는 프로토콜을 적용할 수 있다.

가. 프로토콜의 개념과 3요소

- 1) 프로토콜은 통신을 통제하는 규칙의 집합으로, 무엇을·어떻게·언제 통신할지를 정의하며 구문(Syntax)·의미(Semantics)·타이밍(Timing)의 3요소로 구성된다.
- 2) 프로토콜은 다중화, 주소 설정, 캡슐화, 단편화·재조립, 순서제어, 오류제어, 흐름제어 기능을 통해 안정적인 통신을 보장한다.

나. OSI 7계층과 TCP/IP 모델

- 1) OSI 7계층은 물리-데이터링크-네트워크-전송-세션-표현-응용 계층으로 구성된 이론적 참조 모델이며, TCP/IP는 실제 인터넷에서 사용되는 4계층(네트워크 액세스-인터넷-전송-응용) 실용 모델이다.

[표 3-1] OSI 7계층과 TCP/IP 모델 대응

OSI 7계층	TCP/IP 계층	대표 프로토콜
응용/표현/세션	응용(Application)	HTTP, DNS, FTP, SSH
전송	전송(Transport)	TCP, UDP
네트워크	인터넷(Internet)	IPv4, IPv6, ICMP, ARP
데이터링크/물리	네트워크 액세스	Ethernet, WLAN

다. IP 주소 체계

- 1) IPv4는 32비트 주소를 클래스(A~E) 또는 CIDR 표기로 관리하며, 사설 IP 대역(10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16 등)을 통해 내부망을 구성한다.
- 2) TCP는 연결지향형으로 신뢰성 있는 전송을, UDP는 비연결형으로 빠른 전송을 제공하여 응용 특성에 따라 선택한다.

라. 포트 번호와 서비스

- 1) 포트 번호는 하나의 IP 주소 안에서 여러 서비스를 구분하는 번호로, HTTP는 80, HTTPS는 443, FTP는 21번을 기본으로 사용한다.
- 2) 방화벽은 허용된 포트만 개방하여 불필요한 서비스로의 접근을 차단함으로써 보안을 강화한다.

▶ 실무 Tip

네트워크 장애 발생 시에는 물리 계층(케이블·장비 전원)부터 응용 계층 순으로 아래에서 위로 점검하면 원인을 빠르게 좁힐 수 있다.

학습 2 미들웨어 기초 활용하기

2-1. 미들웨어 파악

학습목표 운영체제와 응용소프트웨어 사이에 위치한 미들웨어의 역할과 기능을 파악할 수 있다.

가. 미들웨어의 개념

1) 미들웨어는 운영체제와 응용소프트웨어 사이에서 통신·트랜잭션·보안 등 공통 기능을 제공하여 응용소프트웨어 개발과 유지보수를 단순화한다.

나. 미들웨어의 주요 기능 분류

1) 분산 시스템 SW(원격 프로시저 호출, 메시지 큐), IT 자원 관리(WAS, DBMS 연동), 서비스 플랫폼(포털, API 게이트웨이), 네트워크 보안(방화벽, VPN) 등으로 분류할 수 있다.

[표 3-2] 미들웨어 기능별 분류

분류	주요 기능	예시
분산 시스템 SW	원격 통신, 메시지 전달	RPC, MOM(메시지 큐)
IT 자원 관리	트랜잭션·자원 연동 관리	WAS(Tomcat), TP-Monitor
서비스 플랫폼	서비스 통합·중계	API Gateway, ESB

다. 네트워크 보안 미들웨어

1) 방화벽, VPN, 침입 탐지 시스템(IDS)과 같은 네트워크 보안 미들웨어는 외부 위협으로부터 내부 시스템을 보호하는 역할을 한다.

2-2. 미들웨어 운용

학습목표 미들웨어를 설치·설정하여 응용소프트웨어와 연동할 수 있다.

가. WAS 운용

1) WAS는 애플리케이션 배포, 커넥션 풀 관리, 세션 클러스터링 기능을 제공하며 대표적으로 Tomcat, JBoss, WebLogic이 있다.

나. 메시지 큐(MQ)

1) 메시지 큐는 시스템 간 비동기 통신을 지원하여 특정 시스템의 장애가 전체 서비스로 전파되는 것을 완화한다.

2) 생산자(Producer)가 큐에 메시지를 적재하면 소비자(Consumer)가 이를 순차적으로 처리하며, 처리 지연이 발생해도 메시지가 유실되지 않고 대기한다.

학습 3 데이터베이스 기초 활용하기

3-1. 데이터베이스 특징 식별

학습목표 관계형 데이터베이스의 개념과 특징을 식별할 수 있다.

가. DBMS의 개념

1) DBMS(Database Management System)는 데이터의 정의·조작·제어를 담당하는 소프트웨어로, 데이터 중복을 최소화하고 무결성과 보안을 보장한다.

나. 관계형 데이터베이스의 구성 요소

1) 테이블(릴레이션), 튜플(행), 속성(열), 기본키(Primary Key), 외래키(Foreign Key)로 구성되며 ERD(개체-관계 다이어그램)로 구조를 표현한다.

2) 스키마는 데이터베이스의 구조·제약 조건을 정의한 것으로, 논리적으로 여러 테이블 간의 관계를 규정한다.

▶ 예시: 학생-수강 테이블 관계

학생(학번, 이름, 학과) — 수강(학번, 과목코드, 성적)

학번은 학생 테이블의 기본키이자 수강 테이블의 외래키로 사용되어 두 테이블을 연결한다.

3-2. 관계형 데이터베이스 테이블 정의

학습목표 요구사항을 바탕으로 테이블을 정의하고 무결성 제약 조건을 설정할 수 있다.

가. 정규화(Normalization)

1) 정규화는 데이터 중복을 줄이고 이상 현상(삽입·갱신·삭제 이상)을 방지하기 위해 테이블을 분해하는 과정으로, 제1정규형부터 제3정규형까지가 실무에서 널리 적용된다.

나. 무결성 제약 조건

1) 개체 무결성은 기본키가 NULL이 될 수 없다는 규칙이며, 참조 무결성은 외래키 값이 참조 테이블에 존재하는 값이어야 한다는 규칙이다.

2) 도메인 무결성은 칼럼에 저장되는 값이 정의된 데이터 타입·범위를 벗어나지 않아야 한다는 규칙이다.

3-3. 관계형 데이터베이스 테이블 조작

학습목표 SQL을 활용하여 테이블 데이터를 삽입·조회·수정·삭제할 수 있다.

가. CRUD와 MVC

- 1) CRUD는 Create·Read·Update·Delete의 약자로 데이터 조작의 기본 단위이며, MVC(Model-View-Controller) 아키텍처에서 Model 계층이 CRUD를 통해 데이터베이스와 상호작용한다.
- 2) 트랜잭션은 CRUD 연산을 하나의 작업 단위로 묶어 전부 성공하거나 전부 취소되도록 보장하여 데이터 일관성을 유지한다.

응용SW 기초 기술 활용 - 학습내용 확인하기

1. 프로토콜과 OSI 7계층

[핵심 요약]

- 가. 프로토콜은 구문·의미·타이밍의 3요소로 구성된다.
- 나. OSI 7계층은 이론적 모델, TCP/IP는 실용 모델이다.

[빈칸 채우기]

- ① 통신 규칙 중 데이터의 구조·형식을 정의하는 요소를 ()이라 한다.
- ② 신뢰성 있는 연결지향 전송을 제공하는 프로토콜은 ()이다.

2. 미들웨어의 역할

[핵심 요약]

- 가. 미들웨어는 운영체제와 응용소프트웨어 사이에 위치한다.
- 나. WAS는 대표적인 IT 자원 관리형 미들웨어이다.

[빈칸 채우기]

- ① 시스템 간 비동기 통신을 지원하는 미들웨어를 ()라 한다.
- ② 애플리케이션을 배포하고 실행하는 미들웨어를 ()라 한다.

3. 데이터베이스 기초와 정규화

[핵심 요약]

- 가. 정규화는 데이터 중복과 이상 현상을 줄이기 위한 과정이다.
- 나. 참조 무결성은 외래키가 참조 테이블에 존재해야 한다는 규칙이다.

[빈칸 채우기]

- ① 데이터베이스 구조를 표현하는 다이어그램을 ()라 한다.
- ② 데이터 조작의 기본 4가지 연산을 통칭하여 ()라 한다.

[정리 문제 - 서술형]

1. OSI 7계층과 TCP/IP 모델의 차이를 설명하시오.
2. 미들웨어가 필요한 이유를 응용소프트웨어 개발 관점에서 서술하시오.
3. 정규화가 필요한 이유를 이상 현상과 연관지어 설명하시오.
4. 기본키와 외래키의 역할 차이를 설명하시오.
5. TCP와 UDP의 차이를 응용 사례를 들어 설명하시오.

[정답]

1. OSI 7계층은 국제표준화기구가 정의한 이론적 참조 모델이며, TCP/IP는 실제 인터넷에서 사용되는 4계층 실용 모델이다.
2. 미들웨어가 없으면 각 응용소프트웨어가 통신·트랜잭션·보안 기능을 개별적으로 구현해야 하므로, 미들웨어가 공통 기능을 제공하여 개발 생산성과 유지보수성을 높인다.
3. 정규화를 하지 않으면 데이터 중복으로 인해 삽입·갱신·삭제 시 일관성이 깨지는 이상 현상이 발생하므로, 테이블을 분해하여 중복을 최소화한다.
4. 기본키는 테이블 내 행을 유일하게 식별하는 역할을 하고, 외래키는 다른 테이블의 기본키를 참조하여 테이블 간 관계를 맺는 역할을 한다.
5. TCP는 신뢰성이 중요한 파일 전송·웹 통신에 사용되고, UDP는 실시간성이 중요한 영상 스트리밍·온라인 게임 등에 사용된다.

응용SW 기초 기술 활용 - 형성평가

1. 프로토콜의 3요소에 해당하지 않는 것은?
① 구문 ② 의미 ③ 타이밍 ④ 용량
2. OSI 7계층 중 전송 계층에 대응하는 TCP/IP 프로토콜은?
① HTTP ② TCP/UDP ③ ARP ④ Ethernet
3. 신뢰성보다 속도를 중시하는 전송 프로토콜은?
① TCP ② UDP ③ FTP ④ SSH
4. 운영체제와 응용소프트웨어 사이에서 공통 기능을 제공하는 것은?
① 컴파일러 ② 미들웨어 ③ 라이브러리 ④ 커널
5. WAS의 주된 역할로 옳은 것은?
① 정적 파일 저장 ② 애플리케이션 실행 및 배포 관리 ③ 이메일 발송 ④ 방화벽 설정
6. 데이터 중복과 이상 현상을 줄이기 위한 과정은?
① 캡슐화 ② 정규화 ③ 다중화 ④ 가상화
7. 외래키가 참조 테이블에 반드시 존재해야 한다는 제약은?
① 개체 무결성 ② 참조 무결성 ③ 도메인 무결성 ④ 키 무결성
8. 데이터베이스 구조를 개체와 관계로 표현하는 다이어그램은?
① UML ② ERD ③ DFD ④ Flowchart

9. CRUD에 해당하지 않는 연산은?

- ① Create ② Read ③ Update ④ Compile

10. 시스템 간 비동기 통신을 지원하는 미들웨어 형태는?

- ① 메시지 큐 ② 컴파일러 ③ 데이터베이스 ④ 텍스트 에디터

11. 사설 IP 대역의 대표적인 예를 하나 쓰시오.

12. 기본키가 NULL이 될 수 없다는 제약 조건의 명칭을 쓰시오.

13. MVC 아키텍처에서 데이터베이스와 상호작용하는 계층의 명칭을 쓰시오.

14. HTTP가 기본으로 사용하는 포트 번호를 쓰시오.

15. 여러 CRUD 연산을 하나의 작업 단위로 묶어 일관성을 보장하는 개념을 쓰시오.

[정답 및 해설]

문항	정답	해설
1	④	프로토콜 3요소는 구문·의미·타이밍이다.
2	②	전송 계층은 TCP/UDP에 대응한다.
3	②	UDP는 비연결형으로 속도가 빠르다.
4	②	미들웨어는 OS와 응용SW 사이에서 공통 기능을 제공한다.
5	②	WAS는 애플리케이션 실행·배포를 관리한다.
6	②	정규화는 중복 최소화와 이상 현상 방지를 위한 과정이다.
7	②	참조 무결성은 외래키의 유효성을 보장한다.
8	②	ERD는 개체-관계 다이어그램이다.
9	④	Compile은 CRUD 연산이 아니다.
10	①	메시지 큐는 비동기 통신을 지원한다.
11	10.0.0.0/8 (또는 172.16.0.0/12, 192.168.0.0/16)	사설 IP 대역 예시이다.
12	개체 무결성	기본키는 NULL을 허용하지 않는다.
13	Model	MVC의 Model 계층이 DB와 상호작용한다.
14	80	HTTP는 기본적으로 80번 포트를 사용한다.
15	트랜잭션	CRUD 연산을 하나의 단위로 묶어 일관성을 보장한다.

4. 개발자 환경 구축 [LM2001020233_23v5]

학습 1 운영체제 기초 활용하기

1-1. 운영체제 식별

학습목표 응용소프트웨어 개발에 필요한 다양한 운영체제의 특징을 식별할 수 있다.

가. 운영체제의 개념과 주요 기능

- 1) 운영체제(OS)는 하드웨어와 소프트웨어 자원을 관리하여 사용자에게 편의성을 제공하는 시스템 소프트웨어로, 처리 능력 향상, 응답시간 단축, 신뢰도 향상, 사용자 인터페이스 제공 등의 기능을 수행한다.
- 2) 주기억장치 관리, CPU 스케줄링, 파일 관리, 명령어 해석·실행도 운영체제의 핵심 기능이다.

나. 운영체제의 종류와 특징

[표 4-1] 주요 운영체제 비교

운영체제	특징	주요 사용 분야
Windows	직관적 GUI, 높은 호환성, 유료 라이선스	개인용 PC, 중소기업 업무용
UNIX	높은 안정성, 대용량 처리	대형 서버
Linux	오픈소스, 무료, 자유로운 수정·배포	서버, 임베디드, 슈퍼컴퓨터
iOS / Android	모바일 전용, 번들형/개방형	스마트폰, 태블릿

다. 운영체제 선정 시 고려사항

- 1) 개발할 응용소프트웨어의 실행 환경, 라이선스 비용, 팀의 숙련도, 지원되는 하드웨어 사양을 종합적으로 고려하여 운영체제를 선정한다.
- 2) 서버용 운영체제는 장시간 무중단 운영과 원격 관리 기능이 중요하므로 안정성과 보안 패치 주기를 우선적으로 검토한다.

1-2. 운영체제 기본 명령어 활용

학습목표 CLI 환경에서 기본 명령어를 사용하여 파일·프로세스를 관리할 수 있다.

가. 파일·디렉터리 명령어

1) 리눅스 계열에서는 ls(목록), cd(이동), cp(복사), mv(이동/이름변경), rm(삭제), mkdir(생성)과 같은 명령어로 파일 시스템을 관리한다.

나. 권한과 프로세스 관리

1) chmod·chown 명령으로 파일 접근 권한과 소유자를 변경하며, ps·top 명령으로 실행 중인 프로세스 상태를 확인한다.

2) kill 명령으로 응답하지 않는 프로세스를 종료할 수 있으며, grep·find 명령으로 파일 내용이나 경로를 신속히 검색한다.

▶ 예시: 자주 쓰는 리눅스 명령어

ls -l : 파일 목록을 상세 정보(권한·소유자·크기)와 함께 표시

chmod 755 파일명 : 소유자에게 읽기·쓰기·실행, 그 외에는 읽기·실행 권한 부여

ps -ef | grep java : 실행 중인 자바 프로세스 검색

1-3. 운영체제 기초 활용

학습목표 운영체제 환경설정과 자원 모니터링을 통해 안정적인 개발 환경을 유지할 수 있다.

가. 환경변수 설정

1) PATH, JAVA_HOME 등 환경변수를 설정하여 명령줄에서 개발 도구를 어디서든 실행할 수 있도록 구성한다.

나. 자원 모니터링

1) CPU·메모리·디스크 사용률을 주기적으로 점검하여 개발 환경의 성능 저하를 예방한다.

2) 로그 파일을 정기적으로 백업·정리하지 않으면 디스크 공간 부족으로 서비스 장애가 발생할 수 있으므로 로그 로테이션 정책을 적용한다.

학습 2 기본 개발 환경 구축하기

2-1. 운영체제 설치

학습목표 요구되는 하드웨어 사양에 맞추어 운영체제를 설치할 수 있다.

가. 설치 사전 준비

- 1) OS 매뉴얼을 통해 최소·권장 하드웨어 사양을 확인하고, 설치 매체(ISO 이미지, USB 부팅 디스크)를 준비한다.

나. 설치 절차

- 1) 파티션 구성 → OS 설치 → 초기 계정·네트워크 설정 → 보안 업데이트 적용 순으로 진행한다.
- 2) 설치 후에는 불필요한 기본 서비스와 계정을 비활성화하여 보안 취약점을 최소화하는 하드닝(Hardening) 작업을 수행한다.

2-2. 개발도구 설치

학습목표 요구사항에 맞는 통합개발환경(IDE)과 형상관리 도구를 설치할 수 있다.

가. 통합개발환경(IDE)

- 1) Eclipse, IntelliJ IDEA, Visual Studio Code 등은 코드 편집, 디버깅, 빌드 기능을 하나의 환경에서 제공하여 개발 생산성을 높인다.

나. 형상관리·빌드 도구

- 1) Git과 같은 분산 형상관리 도구, Maven·Gradle과 같은 빌드 도구를 설치하여 협업과 자동화 빌드 환경을 구축한다.
- 2) 도구 설치 후에는 라이선스와 사내 보안 정책을 준수하는지 확인하고, 팀 전체가 동일한 버전을 사용하도록 표준화한다.

2-3. 개발도구 활용

학습목표 설치된 개발도구를 활용하여 실제 개발 업무를 수행할 수 있다.

가. 플러그인 및 확장 기능

- 1) IDE의 플러그인을 통해 코드 포맷터, 정적 분석기, 프레임워크 지원 기능을 추가하여 개발 편의성을 높인다.

나. 디버깅 환경

- 1) 중단점(Breakpoint), 변수 감시(Watch), 호출 스택 확인 기능을 활용하여 실행 중 오류의 원인을 추적한다.
- 2) 단계별 실행(Step Over, Step Into)을 통해 코드가 실제로 어떤 순서로 동작하는지 확인하며 문제 구간을 좁혀 나간다.

개발자 환경 구축 - 학습내용 확인하기

1. 운영체제의 기능과 종류

[핵심 요약]

- 가. 운영체제는 하드웨어·소프트웨어 자원을 관리하는 시스템 소프트웨어이다.
- 나. Windows·UNIX·Linux·모바일 OS는 각기 다른 특징을 가진다.

[빈칸 채우기]

- ① 오픈소스이며 무료로 자유롭게 수정·배포할 수 있는 운영체제는 ()이다.
- ② 운영체제가 사용자와 시스템 사이에서 제공하는 것을 ()라 한다.

2. 기본 명령어와 환경설정

[핵심 요약]

- 가. CLI 환경에서 ls, cd, cp 등의 명령어로 파일을 관리한다.
- 나. 환경변수를 통해 개발 도구의 실행 경로를 지정한다.

[빈칸 채우기]

- ① 파일 접근 권한을 변경하는 명령어는 ()이다.
- ② 명령줄에서 프로그램 경로를 지정하는 대표적 환경변수는 ()이다.

3. 개발도구 설치와 활용

[핵심 요약]

- 가. IDE는 코드 편집·빌드·디버깅을 통합 제공한다.
- 나. 디버깅 시 중단점과 변수 감시 기능을 활용한다.

[빈칸 채우기]

- ① 코드 편집과 디버깅을 하나의 환경에서 제공하는 도구를 ()라 한다.
- ② 실행을 특정 지점에서 멈추게 하는 디버깅 기능을 ()라 한다.

[정리 문제 - 서술형]

1. Windows, Linux, UNIX 운영체제의 특징을 각각 한 문장으로 비교하시오.
2. IDE를 사용함으로써 얻는 이점을 서술하시오.
3. 개발 환경 구축 시 사전에 확인해야 할 사항을 두 가지 이상 서술하시오.
4. 운영체제 설치 후 수행하는 하드닝(Hardening) 작업의 목적을 서술하시오.
5. 디버깅 시 중단점과 변수 감시 기능이 각각 어떤 역할을 하는지 설명하시오.

[정답]

1. Windows는 GUI 기반의 개인용 OS, Linux는 오픈소스 기반의 서버·임베디드용 OS, UNIX는 안정성이 높은 대형 서버용 OS이다.
2. 코드 작성, 컴파일, 디버깅, 형상관리를 하나의 환경에서 처리할 수 있어 개발 생산성과 오류 대응 속도가 높아진다.
3. 하드웨어 최소·권장 사양, 라이선스 정책, 네트워크·보안 요구사항 등을 사전에 확인해야 한다.
4. 불필요한 기본 서비스·계정을 비활성화하여 보안 취약점을 최소화하기 위함이다.
5. 중단점은 실행을 특정 지점에서 멈추게 하고, 변수 감시는 실행 중 변수 값의 변화를 실시간으로 확인할 수 있게 한다.

개발자 환경 구축 - 형성평가

1. 운영체제의 주요 기능으로 옳지 않은 것은?
① 처리 능력 향상 ② 응답시간 단축 ③ 소스코드 컴파일 최적화만 담당 ④ 파일 관리
2. 오픈소스로 무료 배포가 가능한 운영체제는?
① Windows ② UNIX ③ Linux ④ iOS
3. 리눅스에서 파일 목록을 확인하는 명령어는?
① cd ② ls ③ rm ④ chmod
4. 파일 접근 권한을 변경하는 명령어는?
① chmod ② mkdir ③ cp ④ mv
5. 실행 중인 프로세스 상태를 확인하는 명령어는?
① ps ② rm ③ mv ④ mkdir
6. 코드 편집·빌드·디버깅을 통합 제공하는 도구는?
① 컴파일러 ② IDE ③ 웹 브라우저 ④ 스프레드시트
7. 분산 형상관리 도구의 대표적인 예는?
① Git ② Excel ③ Photoshop ④ Notepad

8. 실행을 특정 지점에서 멈추는 디버깅 기능은?
 ① 중단점 ② 변수 선언 ③ 컴파일 ④ 캐싱
9. 스마트폰·태블릿에 주로 사용되는 운영체제는?
 ① UNIX ② Android ③ 서버용 Linux ④ DOS
10. 개발 도구의 실행 경로를 지정하는 환경변수는?
 ① PATH ② PORT ③ HOST ④ USER
11. 컴퓨터 시스템의 자원을 관리하고 사용자 인터페이스를 제공하는 시스템 소프트웨어를 무엇이라 하는가?
12. Java 개발 시 JDK 경로를 지정하는 대표적인 환경변수를 쓰시오.
13. Maven, Gradle과 같이 컴파일·패키징을 자동화하는 도구를 통칭하여 무엇이라 하는가?
14. 응답하지 않는 프로세스를 종료하는 리눅스 명령어를 쓰시오.
15. 실행을 단계별로 진행하며 코드 동작을 확인하는 디버깅 기법을 무엇이라 하는가?

[정답 및 해설]

문항	정답	해설
1	③	운영체제 기능은 컴파일 최적화에 한정되지 않는다.
2	③	Linux는 오픈소스로 무료 배포가 가능하다.
3	②	ls는 파일 목록을 확인하는 명령어이다.
4	①	chmod는 접근 권한을 변경한다.
5	①	ps는 프로세스 상태를 확인한다.
6	②	IDE는 개발 전 과정을 통합 지원한다.
7	①	Git은 대표적인 분산 형상관리 도구이다.
8	①	중단점은 실행을 특정 위치에서 멈춘다.
9	②	Android는 모바일 전용 운영체제이다.
10	①	PATH는 실행 경로를 지정하는 환경변수이다.
11	운영체제(OS)	하드웨어·소프트웨어 자원을 관리하는 시스템 소프트웨어이다.
12	JAVA_HOME	JDK 설치 경로를 지정하는 환경변수이다.
13	빌드 도구	컴파일·패키징을 자동화하는 도구이다.
14	kill	응답하지 않는 프로세스를 종료하는 명령어이다.
15	단계별 실행(Step Execution)	코드를 한 줄씩 실행하며 동작을 확인하는 기법이다.

학습 1 사용성 테스트 계획하기

1-1. 테스트 기법 선정

학습목표 구현된 UI의 사용성을 검증하기 위해 적합한 테스트 기법을 선정할 수 있다.

가. 사용성 테스트의 개념

- 1) 사용성 테스트는 일반 사용자가 실제 시스템을 사용하도록 하면서 UI에서 발생하는 문제점을 찾아내는 검증 활동이다.

나. 휴리스틱 평가(Heuristic Evaluation)

- 1) 휴리스틱 평가는 사용성 전문가가 이론과 경험을 바탕으로 시스템이 사용성 원칙을 얼마나 준수하는지 판단하는 기법이다.
- 2) 적은 비용으로 빠르게 문제를 발견할 수 있으나, 계량적 자료를 만들기 어렵고 전문가와 실제 사용자의 관점 차이가 존재할 수 있다는 한계가 있다.

다. 그 밖의 사용성 테스트 기법

- 1) 사용자에게 과업을 수행하며 생각을 소리 내어 말하게 하는 씹크 얼라우드(Think Aloud), 두 가지 대안을 비교하는 A/B 테스트, 사용자의 시선 이동을 분석하는 시선 추적(Eye Tracking) 등이 있다.

[표 5-1] 사용성 테스트 기법 비교

기법	특징	장점
휴리스틱 평가	전문가가 원칙 준수 여부 판단	빠르고 저비용
씹크 얼라우드	사용자가 생각을 말하며 과업 수행	실제 사고 과정 파악 가능
A/B 테스트	두 가지 이상의 안을 비교	정량적 비교 가능

라. 카드 소팅(Card Sorting)

- 1) 카드 소팅은 사용자가 콘텐츠 항목이 적힌 카드를 스스로 분류하게 하여 메뉴 구조·정보 분류 체계가 사용자의 인식과 일치하는지 검증하는 기법이다.

1-2. 테스트 환경 구축

학습목표 사용성 테스트에 필요한 장비와 환경을 구축할 수 있다.

가. 테스트 환경 구성 요소

- 1) 사용성 테스트 룸, 화면·음성 녹화 장비, 시간 기록기, 관찰용 매직미러 등을 활용하여 실제와 유사한 환경을 구성한다.

나. 참여자 모집

- 1) 실제 시스템을 사용할 대상 사용자의 속성(연령, 숙련도 등)에 가까운 참여자를 모집해야 테스트 결과의 타당성이 높아진다.
- 2) 일반적으로 5명 내외의 참여자로도 사용성 문제의 상당수를 발견할 수 있다고 알려져 있어, 소규모 반복 테스트가 효율적이다.

1-3. 사용성 테스트 계획서 작성

학습목표 테스트 목적·대상·시나리오를 포함한 계획서를 작성할 수 있다.

가. 계획서 구성 항목

- 1) 목표 고객, 테스트 목적, 과업 시나리오, 평가 지표, 일정, 참여자 수 등을 계획서에 명시한다.

나. 과업 시나리오 설계

- 1) 실제 사용 맥락을 반영한 구체적 과업(예: 회원가입 후 상품 결제하기)을 설계하여 사용자가 자연스럽게 UI를 조작하도록 유도한다.
- 2) 과업은 특정 UI 요소를 직접 지칭하지 않고 목표만 제시해야 사용자가 스스로 경로를 탐색하는 과정에서 실제 사용성 문제가 드러난다.

학습 2 사용성 테스트 수행하기

2-1. 사용성 테스트 수행

학습목표 계획서에 따라 사용성 테스트를 진행하고 결과를 기록할 수 있다.

가. 테스트 진행 절차

- 1) 참여자에게 과업을 안내 → 과업 수행 관찰·녹화 → 사후 인터뷰 순으로 진행하며, 진행자는 유도 질문을 피해야 한다.

나. 정량·정성 데이터 수집

- 1) 과업 성공률, 소요 시간과 같은 정량 데이터와 사용자의 발화·표정과 같은 정성 데이터를 함께 수집한다.
- 2) 화면 녹화와 함께 사후 인터뷰를 진행하면 사용자가 특정 행동을 한 이유(맥락)까지 파악할 수 있어 분석의 깊이가 더해진다.

2-2. 평가 분석서 작성 및 이슈 도출

학습목표 테스트 결과를 분석하여 사용성 이슈를 도출하고 심각도를 판단할 수 있다.

가. 이슈 도출

- 1) 반복적으로 관찰된 문제 행동, 과업 실패 지점을 UI 이슈로 정리하고 원인을 분석한다.

나. 심각도 분류

- 1) 이슈는 사용자 과업 수행에 미치는 영향에 따라 치명적·중대·경미로 분류하여 개선 우선순위를 정한다.
- 2) 동일한 이슈가 여러 참여자에게서 반복적으로 관찰될수록 우선순위를 높게 책정한다.

▶ 예시: 평가 분석서 구성 항목

- 참여자 정보(익명화) / 과업별 성공·실패 여부 / 소요 시간
- 관찰된 문제 행동과 발화 내용 / 이슈 요약 및 심각도

학습 3 테스트 결과 보고하기

3-1. UI 개선방안 및 수정계획 수립

학습목표 도출된 이슈에 대한 개선방안을 제시하고 수정 계획을 수립할 수 있다.

가. 개선방안 도출

- 1) 이슈별로 UI 설계 변경, 문구 수정, 인터랙션 개선 등 구체적인 개선안을 제시한다.

나. 수정 계획 수립

- 1) 개선 우선순위와 개발 일정을 고려하여 단계적 수정 계획을 수립한다.

3-2. UI 개선 결과보고서 공유

학습목표 개선 결과를 정리한 보고서를 작성하고 이해관계자와 공유할 수 있다.

가. 보고서 구성

- 1) 테스트 개요, 주요 이슈, 개선 전후 비교, 향후 계획을 포함하여 보고서를 작성한다.

나. 공유 및 피드백

- 1) 이해관계자에게 보고서를 공유하고 추가 의견을 반영하여 후속 개선 활동을 계획한다.

1. 사용성 테스트 기법

[핵심 요약]

- 가. 휴리스틱 평가는 전문가가 원칙 준수를 판단하는 기법이다.
- 나. 씹크 얼라우드는 사용자가 생각을 말하며 과업을 수행하는 기법이다.

[빈칸 채우기]

- ① 전문가가 이론과 경험으로 사용성 문제를 판단하는 기법을 () 평가라 한다.
- ② 사용자가 생각을 소리 내어 말하며 과업을 수행하는 기법을 ()라 한다.

2. 테스트 계획과 수행

[핵심 요약]

- 가. 계획서에는 목적·대상·시나리오·평가지표가 포함된다.
- 나. 테스트 진행자는 유도 질문을 피해야 한다.

[빈칸 채우기]

- ① 사용자가 수행할 구체적 과업을 ()라 한다.
- ② 과업 성공률, 소요 시간과 같은 데이터를 () 데이터라 한다.

3. 이슈 도출과 보고

[핵심 요약]

- 가. 이슈는 치명적·중대·경미로 심각도를 분류한다.
- 나. 결과보고서는 이슈, 개선방안, 향후 계획을 포함한다.

[빈칸 채우기]

- ① 반복적으로 발견된 문제 행동을 정리한 것을 UI ()라 한다.
- ② 개선 결과를 정리하여 공유하는 문서를 ()라 한다.

[정리 문제 - 서술형]

1. 휴리스틱 평가의 장점과 단점을 각각 서술하시오.
2. 사용성 테스트 참여자를 모집할 때 고려해야 할 사항을 서술하시오.
3. UI 이슈의 심각도를 분류하는 기준과 그 목적을 설명하시오.
4. 과업 시나리오를 설계할 때 특정 UI 요소를 직접 지칭하지 않아야 하는 이유를 서술하시오.
5. 정량 데이터와 정성 데이터의 차이를 예를 들어 설명하시오.

[정답]

1. 장점은 적은 비용으로 빠르게 문제를 발견할 수 있다는 것이고, 단점은 계량적 자료 확보가 어렵고 전문가와 실제 사용자의 시각 차이가 있을 수 있다는 것이다.

2. 실제 시스템을 사용할 대상 사용자의 연령·숙련도 등 속성에 가까운 참여자를 모집해야 테스트 결과의 타당성이 높아진다.
3. 이슈가 과업 수행에 미치는 영향 정도(치명적·중대·경미)에 따라 분류하며, 이를 통해 제한된 자원으로 개선 우선순위를 합리적으로 정할 수 있다.
4. 특정 요소를 지칭하면 사용자가 그 요소만 찾게 되어 실제 탐색 과정에서 드러나는 사용성 문제를 관찰할 수 없기 때문이다.
5. 정량 데이터는 과업 성공률·소요 시간과 같이 수치로 측정되는 자료이고, 정성 데이터는 사용자의 발화·표정과 같이 수치화하기 어려운 관찰 자료이다.

UI 테스트 - 형성평가

1. 사용성 테스트의 목적으로 가장 적절한 것은?
 ① 소스코드 성능 측정 ② UI 사용 중 발생하는 문제점 발견 ③ 데이터베이스 정규화 ④ 네트워크 속도 측정
2. 전문가가 이론과 경험으로 사용성을 평가하는 기법은?
 ① 씽크 얼라우드 ② 휴리스틱 평가 ③ A/B 테스트 ④ 코드 인스펙션
3. 사용자가 생각을 말하며 과업을 수행하는 기법은?
 ① 휴리스틱 평가 ② 씽크 얼라우드 ③ 시선 추적 ④ 카드 소팅
4. 사용성 테스트 계획서에 포함되지 않아도 되는 항목은?
 ① 목표 고객 ② 과업 시나리오 ③ 소스코드 컴파일 옵션 ④ 평가 지표
5. 테스트 진행자가 지켜야 할 태도로 옳은 것은?
 ① 정답을 미리 알려준다 ② 유도 질문을 피한다 ③ 참여자를 재촉한다 ④ 과업을 생략한다
6. 과업 성공률, 소요 시간과 같은 데이터의 성격은?
 ① 정성 데이터 ② 정량 데이터 ③ 정적 데이터 ④ 결함 데이터
7. UI 이슈의 심각도 분류에 해당하지 않는 것은?
 ① 치명적 ② 중대 ③ 경미 ④ 무한
8. 이슈 개선 우선순위를 정할 때 가장 중요하게 고려할 것은?
 ① 이슈의 심각도와 영향 ② 개발자의 선호 ③ 코드 라인 수 ④ 서버 위치
9. 사용성 테스트 결과보고서에 포함되지 않는 것은?
 ① 테스트 개요 ② 주요 이슈 ③ 향후 계획 ④ 서버 IP 대역
10. 실제 사용 맥락을 반영해 사용자가 수행할 구체적 행동을 설계한 것은?
 ① 과업 시나리오 ② 결함 목록 ③ 소스 코드 ④ ERD
11. 사용자의 시선 이동을 분석하는 사용성 테스트 기법의 명칭을 쓰시오.
12. 두 가지 이상의 UI 안을 비교하여 더 나은 안을 선택하는 기법을 쓰시오.
13. 사용성 테스트에서 참여자의 발화·표정과 같은 자료의 성격을 쓰시오.
14. 사용자가 카드를 스스로 분류하게 하여 메뉴 구조를 검증하는 기법을 쓰시오.
15. 사용성 테스트에 필요한 최소 참여자 규모에 대한 통념상의 인원수를 쓰시오.

[정답 및 해설]

문항	정답	해설
1	②	사용성 테스트는 UI 문제점 발견이 목적이다.
2	②	휴리스틱 평가는 전문가 판단 기반 기법이다.
3	②	씹크 얼라우드는 생각을 말하며 과업 수행한다.
4	③	컴파일 옵션은 계획서 항목이 아니다.
5	②	진행자는 유도 질문을 피해야 한다.
6	②	성공률·소요시간은 정량 데이터이다.
7	④	무한은 심각도 분류 기준이 아니다.
8	①	심각도와 영향이 우선순위 결정 기준이다.
9	④	서버 IP는 보고서 항목이 아니다.
10	①	과업 시나리오의 사용자의 구체적 행동을 설계한 것이다.
11	시선 추적(Eye Tracking)	시선 이동을 분석하는 기법이다.
12	A/B 테스트	두 안을 비교하는 정량적 기법이다.
13	정성 데이터	발화·표정 등은 정성적 자료이다.
14	카드 소팅(Card Sorting)	카드 분류로 메뉴 구조를 검증하는 기법이다.
15	5명 내외	소규모 참여자라도 다수의 사용성 문제를 발견할 수 있다.

6. 프로그래밍 언어 활용 [LM2001020231_23v5]

학습 1 구조적 프로그래밍 언어 활용하기

1-1. 구조적 프로그래밍 설계

학습목표 응용소프트웨어 개발을 위해 프로그램 설계서를 확인하고 구조적으로 설계할 수 있다.

가. 구조적 프로그래밍의 개념

1) 구조적 프로그래밍은 절차적 프로그래밍을 기반으로 순차·선택·반복의 세 가지 제어 구조만을 사용하여 프로그램 흐름을 명확하게 만드는 기법이다.

나. 하향식 설계(Top-Down Design)

1) 전체 문제를 상위 모듈에서 하위 모듈로 점진적으로 세분화하여 설계하는 방식으로, 모듈 간 결합도는 낮추고 응집도는 높이는 것을 목표로 한다.

[표 6-1] 구조적 프로그래밍의 3대 제어 구조

제어 구조	설명	예시 문법(의사코드)
순차	명령을 작성한 순서대로 실행	A; B; C;
선택	조건에 따라 실행 경로 분기	IF 조건 THEN A ELSE B
반복	조건을 만족하는 동안 반복 실행	WHILE 조건 DO A

다. 순서도와 의사코드

1) 순서도(Flowchart)는 처리 흐름을 도형으로 시각화한 것이고, 의사코드(Pseudo Code)는 특정 언어 문법에 얽매이지 않고 자연어에 가깝게 논리를 기술한 것이다. 두 가지 모두 코딩 이전에 설계를 검증하는 데 활용된다.

1-2. 구조적 프로그래밍 언어 활용

학습목표 C언어 등 구조적 프로그래밍 언어의 문법을 활용하여 프로그램을 구현할 수 있다.

가. 변수와 자료형

1) int, float, char 등 기본 자료형을 선언하여 메모리에 데이터를 저장하고, 배열·구조체로 복합 데이터를 표현한다.

나. 함수와 모듈화

1) 반복되는 기능을 함수로 분리하면 코드 재사용성과 가독성이 높아지며, 함수 간 매개 변수 전달로 데이터를 주고받는다.

2) 지역 변수는 함수 내부에서만 유효하고, 전역 변수는 프로그램 전체에서 접근 가능하므로 의도치 않은 값 변경을 막기 위해 지역 변수 사용을 우선한다.

학습 2 객체지향 프로그래밍 언어 활용하기

2-1. 객체지향 프로그래밍 설계

학습목표 객체지향 개념에 따라 클래스와 객체를 설계할 수 있다.

가. 객체지향 프로그래밍(OOP)의 개념

1) OOP는 프로그램을 객체(Object)들의 상호작용으로 구성하는 패러다임으로, 클래스는 객체를 만들기 위한 설계도 역할을 한다.

나. OOP의 4대 특성

[표 6-2] 객체지향 프로그래밍의 4대 특성

특성	설명
캡슐화(Encapsulation)	데이터와 메서드를 하나로 묶고 내부 구현을 숨김
상속(Inheritance)	상위 클래스의 속성·메서드를 하위 클래스가 물려받음
다형성(Polymorphism)	동일한 이름의 메서드가 객체에 따라 다르게 동작
추상화(Abstraction)	공통 특성을 추출하여 단순화된 모델로 표현

다. 클래스 다이어그램

1) UML 클래스 다이어그램은 클래스의 속성·메서드와 클래스 간 관계(연관, 상속, 의존)를 시각적으로 표현하여 설계 단계에서 구조를 검토하는 데 활용된다.

2-2. 객체지향 프로그래밍 언어 활용

학습목표 Java 등 객체지향 언어의 문법을 활용하여 클래스를 구현할 수 있다.

가. 클래스와 인스턴스

1) 클래스로부터 생성된 실체를 인스턴스(객체)라 하며, new 연산자 등을 통해 메모리에 할당한다.

나. 접근 제어자와 인터페이스

1) public·private·protected 접근 제어자로 캡슐화를 구현하며, 인터페이스를 통해 클래스 간 규약을 정의한다.

2) 인터페이스는 메서드의 시그니처만 정의하고 구현은 각 클래스에 위임하므로, 서로 다른 클래스가 동일한 규약을 따르도록 강제할 수 있다.

학습 3 스크립트 언어 활용하기

3-1. 스크립트 언어 설계

학습목표 스크립트 언어의 특징을 이해하고 요구사항에 맞는 처리 로직을 설계할 수 있다.

가. 스크립트 언어의 개념

- 1) 스크립트 언어는 별도의 컴파일 없이 인터프리터가 소스를 한 줄씩 해석·실행하는 언어로, Python·JavaScript가 대표적이다.

나. 스크립트 언어의 특징

- 1) 문법이 간결하고 동적 타이핑을 지원하여 빠른 프로토타이핑에 적합하지만, 컴파일 언어보다 실행 속도가 느릴 수 있다.
- 2) 동적 타이핑 언어는 변수의 자료형을 실행 시점에 결정하므로 유연하지만, 실수로 잘못된 타입을 전달해도 실행 전에는 오류를 발견하기 어렵다는 단점이 있다.

3-2. 스크립트 언어 활용

학습목표 스크립트 언어를 활용하여 웹 페이지 동적 기능이나 자동화 스크립트를 구현할 수 있다.

가. 클라이언트 스크립트

- 1) JavaScript는 브라우저에서 실행되어 DOM 조작, 이벤트 처리 등 동적인 웹 UI를 구현한다.

나. 서버·자동화 스크립트

- 1) Python은 데이터 처리, 업무 자동화, 서버 측 로직 구현 등에 널리 사용되며 풍부한 표준 라이브러리를 제공한다.
- 2) 반복적인 사무 작업(파일 정리, 보고서 생성 등)을 스크립트로 자동화하면 처리 시간을 크게 단축할 수 있다.

1. 구조적 프로그래밍

[핵심 요약]

가. 구조적 프로그래밍은 순차·선택·반복의 제어 구조를 사용한다.

나. 하향식 설계는 상위에서 하위로 점진적으로 세분화한다.

[빈칸 채우기]

- ① 조건에 따라 실행 경로를 나누는 제어 구조를 ()라 한다.
- ② 전체 문제를 점진적으로 세분화하는 설계 방식을 () 설계라 한다.

2. 객체지향 프로그래밍

[핵심 요약]

가. 객체지향의 4대 특성은 캡슐화, 상속, 다형성, 추상화이다.

나. 클래스는 객체를 만들기 위한 설계도이다.

[빈칸 채우기]

- ① 상위 클래스의 속성을 하위 클래스가 물려받는 특성을 ()이라 한다.
- ② 동일한 메서드가 객체마다 다르게 동작하는 특성을 ()이라 한다.

3. 스크립트 언어

[핵심 요약]

가. 스크립트 언어는 컴파일 없이 인터프리터가 해석·실행한다.

나. JavaScript는 브라우저에서 동적 UI를 구현한다.

[빈칸 채우기]

- ① 소스를 한 줄씩 해석하며 실행하는 프로그램을 ()라 한다.
- ② 브라우저에서 실행되어 동적 UI를 구현하는 대표 언어는 ()이다.

[정리 문제 - 서술형]

1. 구조적 프로그래밍의 3대 제어 구조를 나열하고 각각 설명하시오.
2. 객체지향 프로그래밍의 4대 특성을 서술하시오.
3. 컴파일 언어와 스크립트(인터프리터) 언어의 차이를 설명하시오.
4. 지역 변수와 전역 변수의 차이와 지역 변수를 우선 사용해야 하는 이유를 서술하시오.
5. 인터페이스가 여러 클래스에 공통 규약을 강제할 수 있는 이유를 설명하시오.

[정답]

1. 순차(명령을 순서대로 실행), 선택(조건에 따라 분기), 반복(조건을 만족하는 동안 반복 실행)이다.
2. 캡슐화(데이터·메서드 은닉), 상속(속성·메서드 물려받기), 다형성(동일 메서드의 다른 동작), 추상화(공

통 특성 단순화)이다.

3. 컴파일 언어는 실행 전 전체를 기계어로 변환하여 실행 속도가 빠르지만 이식성이 낮고, 스크립트 언어는 인터프리터가 한 줄씩 해석·실행하여 이식성과 개발 속도는 높지만 실행 속도는 상대적으로 느리다.
4. 지역 변수는 함수 내부에서만 유효해 다른 코드에 영향을 주지 않지만, 전역 변수는 프로그램 전체에서 접근 가능해 의도치 않은 값 변경이 발생할 수 있으므로 지역 변수를 우선 사용한다.
5. 인터페이스는 메서드의 시그니처만 정의하고 구현은 각 클래스에 위임하므로, 이를 구현하는 모든 클래스가 동일한 메서드 집합을 반드시 갖추도록 강제할 수 있다.

프로그래밍 언어 활용 - 형성평가

1. 구조적 프로그래밍의 제어 구조에 해당하지 않는 것은?
① 순차 ② 선택 ③ 반복 ④ 캡슐화
2. 상위 모듈에서 하위 모듈로 세분화하는 설계 방식은?
① 상향식 설계 ② 하향식 설계 ③ 객체지향 설계 ④ 이벤트 설계
3. 객체를 만들기 위한 설계도 역할을 하는 것은?
① 인스턴스 ② 클래스 ③ 함수 ④ 배열
4. 데이터와 메서드를 하나로 묶어 내부 구현을 숨기는 특성은?
① 상속 ② 다형성 ③ 캡슐화 ④ 추상화
5. 상위 클래스의 속성·메서드를 물려받는 특성은?
① 캡슐화 ② 상속 ③ 다형성 ④ 오버플로
6. 동일한 이름의 메서드가 객체에 따라 다르게 동작하는 특성은?
① 캡슐화 ② 상속 ③ 다형성 ④ 정규화
7. 별도 컴파일 없이 한 줄씩 해석·실행되는 언어는?
① 컴파일 언어 ② 스크립트 언어 ③ 어셈블리어 ④ 기계어
8. 브라우저에서 실행되어 동적 UI를 구현하는 대표 언어는?
① C언어 ② Java ③ JavaScript ④ SQL
9. 접근 제어자에 해당하지 않는 것은?
① public ② private ③ protected ④ dynamic
10. 함수로 기능을 분리했을 때의 장점으로 옳은 것은?
① 코드 중복 증가 ② 재사용성과 가독성 향상 ③ 실행 속도 무조건 저하 ④ 변수 개수 증가
11. 클래스로부터 생성된 실체를 무엇이라 하는가?
12. 공통된 특성을 추출하여 단순화된 모델로 표현하는 객체지향 특성을 쓰시오.
13. 데이터 처리, 자동화 스크립트 작성에 널리 쓰이는 대표 스크립트 언어를 쓰시오.
14. 함수 내부에서만 유효한 변수를 무엇이라 하는가?
15. 처리 흐름을 도형으로 시각화한 설계 산출물을 무엇이라 하는가?

[정답 및 해설]

문항	정답	해설
1	④	캡슐화는 객체지향 특성이다.
2	②	하향식 설계는 상위→하위로 세분화한다.
3	②	클래스는 객체의 설계도이다.
4	③	캡슐화는 데이터·메서드를 묶어 은닉한다.
5	②	상속은 속성·메서드를 물려받는 특성이다.
6	③	다형성은 동일 메서드의 다른 동작을 의미한다.
7	②	스크립트 언어는 인터프리터가 해석·실행한다.
8	③	JavaScript는 브라우저 동적 UI 구현에 쓰인다.
9	④	dynamic은 접근 제어자가 아니다.
10	②	함수 분리는 재사용성·가독성을 높인다.
11	인스턴스(객체)	클래스로부터 생성된 실체이다.
12	추상화	공통 특성을 단순화하여 표현하는 특성이다.
13	Python	데이터 처리·자동화에 널리 쓰이는 언어이다.
14	지역 변수	함수 내부에서만 유효한 변수이다.
15	순서도(Flowchart)	처리 흐름을 도형으로 시각화한 것이다.

7. 화면 구현 [LM2001020225_23v6]

학습 1 UI 설계 확인하기

1-1. UI 설계 내용 확인

학습목표 설계된 화면과 품의 흐름을 확인하여 구현에 반영할 제약사항을 파악할 수 있다.

가. UI(User Interface)의 개념

- 1) UI는 사용자가 시스템과 상호작용할 때 편리성과 가독성을 높이기 위한 화면·입력 요소의 집합이다.

나. UI 설계서 검토

- 1) 화면 정의서, 스토리보드, 와이어프레임을 통해 화면 구성 요소, 입력 항목, 유효성 규칙, 예외 처리 방식을 사전에 확인한다.
- 2) 설계서와 실제 요구사항이 다를 경우 구현 전 담당 기획자·디자이너와 협의하여 불명확한 부분을 명확히 해야 재작업을 줄일 수 있다.

1-2. UI 메뉴 구조 확인

학습목표 설계된 메뉴 구조와 화면 간 이동 흐름(내비게이션)을 확인할 수 있다.

가. 정보 구조(IA)와 내비게이션

- 1) 사이트맵을 통해 메뉴의 계층 구조를 확인하고, 화면 간 이동 경로가 사용자의 업무 흐름과 일치하는지 점검한다.
- 2) 현재 위치를 표시하는 브레드크럼(Breadcrumb), 자주 쓰는 기능으로의 단축 경로 등을 설계에 반영하면 사용자의 탐색 부담을 줄일 수 있다.

학습 2 UI 구현하기

2-1. UI 설계 구현

학습목표 HTML·CSS를 활용하여 설계된 화면을 마크업하고 스타일링할 수 있다.

가. HTML/CSS 기본 구조

- 1) HTML은 화면의 구조(요소·콘텐츠)를 정의하고, CSS는 레이아웃과 시각적 스타일을 지정한다.

나. 반응형 웹과 웹 표준

- 1) 다양한 화면 크기에 대응하기 위해 미디어 쿼리를 활용한 반응형 레이아웃을 적용하며, W3C 웹 표준을 준수하여 브라우저 간 호환성을 높인다.

다. 웹 접근성(Web Accessibility)

- 1) 시각·청각 등 신체적 제약이 있는 사용자도 콘텐츠를 이용할 수 있도록 대체 텍스트, 키보드 내비게이션 등 접근성 지침을 준수한다.
- 2) 색상만으로 정보를 전달하지 않고, 명도 대비를 충분히 확보하며, 스크린리더가 콘텐츠를 올바르게 읽을 수 있도록 시맨틱 태그를 사용한다.

▶ 예시: 웹 접근성 준수 사례

- 이미지에는 alt 속성으로 대체 텍스트 제공
- 폼 입력 항목에는 label 태그로 명확한 레이블 연결
- 마우스 없이 Tab 키만으로 모든 기능에 접근 가능하도록 구현

2-2. UI 제어 구현

학습목표 자바스크립트를 활용하여 사용자 이벤트에 반응하는 동적 화면을 구현할 수 있다.

가. 이벤트 처리

- 1) 클릭, 입력, 포커스 등 사용자 이벤트를 감지하여 지정된 함수(이벤트 핸들러)가 실행 되도록 구현한다.

나. 입력값 유효성 검사

- 1) 필수 입력 여부, 형식(이메일·전화번호 등)을 자바스크립트로 검증하여 잘못된 데이터가 서버로 전송되지 않도록 한다.
- 2) 클라이언트 측 검증은 사용자 경험을 개선하지만 우회될 수 있으므로, 서버 측에서도 동일한 검증을 반드시 재수행해야 한다.

2-3. UI 테스트 설계

학습목표 구현된 화면을 검증하기 위한 테스트 케이스를 설계할 수 있다.

가. 웹 표준·웹 호환성(Cross Browsing) 검사

- 1) W3C Markup Validator 등 검증 도구로 HTML·CSS 유효성을 검사하고, 주요 브라우저에서 동일하게 동작하는지 확인한다.

나. 테스트 케이스 작성

- 1) 입력 항목별 정상·예외 입력값을 정의하여 화면이 설계된 대로 동작하는지 검증하는 테스트 케이스를 작성한다.
- 2) 화면 해상도·기기(PC, 태블릿, 모바일)별로 레이아웃이 깨지지 않는지 확인하는 반응형 테스트도 함께 수행한다.

[표 7-1] 화면 구현 품질 점검 항목

점검 항목	점검 내용
유효성 검사	HTML/CSS 문법 오류 여부 검증
웹 접근성	대체 텍스트, 키보드 접근성 등 준수 여부
웹 호환성	주요 브라우저(Chrome, Edge 등)에서 동일 동작 여부

1. UI 설계 확인

[핵심 요약]

- 가. UI 설계서를 통해 화면 구성과 유효성 규칙을 사전에 확인한다.
- 나. 사이트맵으로 메뉴의 계층 구조를 점검한다.

[빈칸 채우기]

- ① 사용자가 시스템과 상호작용하는 화면·입력 요소를 통칭하여 ()라 한다.
- ② 메뉴의 계층 구조를 한눈에 보여주는 문서를 ()이라 한다.

2. HTML/CSS와 접근성

[핵심 요약]

- 가. HTML은 구조, CSS는 스타일을 담당한다.
- 나. 웹 접근성은 신체적 제약이 있는 사용자도 이용할 수 있도록 하는 것이다.

[빈칸 채우기]

- ① 화면 구조를 정의하는 언어를 ()이라 한다.
- ② 다양한 사용자가 콘텐츠를 이용할 수 있도록 하는 원칙을 ()이라 한다.

3. UI 제어와 테스트

[핵심 요약]

- 가. 자바스크립트로 이벤트 처리와 유효성 검사를 구현한다.
- 나. 웹 호환성 검사로 여러 브라우저에서의 동작을 확인한다.

[빈칸 채우기]

- ① 클릭·입력 등 사용자 동작을 감지하는 것을 ()라 한다.
- ② 여러 브라우저에서 동일하게 동작하는지 확인하는 것을 ()이라 한다.

[정리 문제 - 서술형]

- 1. HTML과 CSS의 역할 차이를 설명하시오.
- 2. 웹 접근성을 고려해야 하는 이유를 서술하시오.
- 3. UI 테스트 설계 시 확인해야 할 항목을 두 가지 이상 서술하시오.
- 4. 클라이언트 측 유효성 검사만으로 충분하지 않은 이유를 서술하시오.
- 5. 브레드크럼(Breadcrumb)이 사용자에게 어떤 도움을 주는지 설명하시오.

[정답]

- 1. HTML은 화면의 구조와 콘텐츠를 정의하고, CSS는 레이아웃과 시각적 스타일을 지정한다.
- 2. 시각·청각 등 신체적 제약이 있는 사용자를 포함한 모든 사용자가 동등하게 콘텐츠를 이용할 수 있도록

록 하기 위함이다.

3. HTML/CSS 유효성, 웹 접근성 준수 여부, 여러 브라우저에서의 동일 동작(웹 호환성) 등을 확인해야 한다.
4. 클라이언트 측 검증은 개발자 도구 등으로 우회될 수 있으므로, 서버 측에서도 동일한 검증을 재수행해야 데이터 무결성을 보장할 수 있다.
5. 현재 페이지의 위치를 상위 메뉴부터 계층적으로 보여주어, 사용자가 자신의 위치를 파악하고 상위 화면으로 쉽게 이동할 수 있도록 돕는다.

화면 구현 - 형성평가

1. 화면의 구조와 콘텐츠를 정의하는 언어는?
① CSS ② HTML ③ SQL ④ UML
2. 레이아웃과 시각적 스타일을 담당하는 언어는?
① HTML ② CSS ③ Java ④ XML
3. 다양한 화면 크기에 대응하는 레이아웃 기법은?
① 고정형 레이아웃 ② 반응형 레이아웃 ③ 정적 레이아웃 ④ 단일형 레이아웃
4. 신체적 제약이 있는 사용자도 콘텐츠를 이용할 수 있도록 하는 원칙은?
① 웹 표준 ② 웹 접근성 ③ 웹 호환성 ④ 웹 보안
5. 여러 브라우저에서 동일하게 동작하는지 확인하는 것은?
① 웹 접근성 ② 웹 호환성 ③ 웹 보안 ④ 웹 성능
6. 클릭·입력 등 사용자 동작을 감지하여 처리하는 것은?
① 이벤트 처리 ② 정규화 ③ 캡슐화 ④ 형상관리
7. 입력 형식(이메일 등)이 올바른지 검사하는 것은?
① 유효성 검사 ② 정적 테스트 ③ 회귀 테스트 ④ 부하 테스트
8. 메뉴의 계층 구조를 표현하는 문서는?
① ERD ② 사이트맵 ③ 순서도 ④ 클래스 다이어그램
9. HTML/CSS 문법 오류를 검사하는 도구는?
① W3C Markup Validator ② JUnit ③ Git ④ Maven
10. 화면 구현 시 참고하는 설계 산출물이 아닌 것은?
① 화면 정의서 ② 와이어프레임 ③ 스토리보드 ④ ERD
11. 사용자와 시스템 간 상호작용을 위한 화면·입력 요소를 통칭하는 용어를 쓰시오.
12. 자바스크립트로 사용자 동작에 반응하는 함수를 무엇이라 하는가?
13. 미디어 쿼리를 활용해 화면 크기에 대응하는 레이아웃 기법을 쓰시오.
14. 이미지에 대체 텍스트를 지정하는 HTML 속성을 쓰시오.
15. 현재 페이지 위치를 계층적으로 표시하는 내비게이션 요소를 쓰시오.

[정답 및 해설]

문항	정답	해설
1	②	HTML은 구조·콘텐츠를 정의한다.
2	②	CSS는 스타일을 담당한다.
3	②	반응형 레이아웃은 다양한 화면 크기에 대응한다.
4	②	웹 접근성은 신체적 제약 사용자를 고려한다.
5	②	웹 호환성은 여러 브라우저 동일 동작을 의미한다.
6	①	이벤트 처리는 사용자 동작 감지·처리이다.
7	①	유효성 검사는 입력 형식을 검증한다.
8	②	사이트맵은 메뉴 계층 구조를 표현한다.
9	①	W3C Markup Validator는 문법 오류를 검사한다.
10	④	ERD는 데이터베이스 설계 산출물이다.
11	U I (U s e r Interface)	사용자와 시스템의 상호작용 요소이다.
12	이벤트 핸들러	사용자 동작에 반응하는 함수이다.
13	반응형 웹(반응형 레이아웃)	미디어 쿼리로 화면 크기에 대응한다.
14	alt	이미지의 대체 텍스트를 지정하는 속성이다.
15	브레드크럼 (Breadcrumb)	현재 위치를 계층적으로 표시하는 요소이다.

학습 1 기본 SQL 작성하기

1-1. 테이블의 데이터 사전 조회

학습목표 데이터 사전을 조회하여 테이블·칼럼·제약 조건 등 객체 정보를 확인할 수 있다.

가. 데이터 사전(Data Dictionary)

- 1) 데이터 사전은 테이블, 인덱스, 뷰 등 데이터베이스 객체에 대한 메타데이터를 저장하며, 시스템 뷰를 조회하여 확인할 수 있다.

나. 데이터베이스 객체

- 1) 오브젝트(Object)는 테이블, 뷰, 인덱스, 제약 조건 등 DBMS가 관리하는 논리적 단위를 말한다.

1-2. 기본적인 SQL 작성

학습목표 DDL·DML·DCL의 기본 명령문을 작성하여 데이터베이스를 정의·조작할 수 있다.

가. DDL(Data Definition Language)

- 1) DDL은 테이블 등 오브젝트의 구조를 정의하는 언어로 CREATE(생성), ALTER(변경), DROP(삭제) 명령어로 구성된다.

나. DML(Data Manipulation Language)

- 1) DML은 데이터를 조작하는 언어로 SELECT(조회), INSERT(삽입), UPDATE(수정), DELETE(삭제) 명령어로 구성된다.

다. DCL(Data Control Language)

- 1) DCL은 데이터 접근 권한과 트랜잭션을 제어하는 언어로 GRANT(권한 부여), REVOKE(권한 회수), COMMIT(확정), ROLLBACK(취소)이 있다.

[표 8-1] SQL 명령어 분류

구분	주요 명령어	기능
DDL	CREATE, ALTER, DROP	테이블 등 객체 구조 정의
DML	SELECT, INSERT, UPDATE, DELETE	데이터 조회 및 조작
DCL	GRANT, REVOKE, COMMIT, ROLLBACK	권한 및 트랜잭션 제어

라. 무결성 제약 조건

1) 기본키(Primary Key)는 행을 유일하게 식별하며 NULL을 허용하지 않고, 외래키(Foreign Key)는 다른 테이블의 기본키를 참조하여 참조 무결성을 보장한다.

2) UNIQUE 제약은 칼럼 값의 중복을 허용하지 않고, CHECK 제약은 지정된 조건을 만족하는 값만 저장되도록 제한한다.

▶ 예시: 테이블 생성 DDL

```
CREATE TABLE 학생 (
    학번 VARCHAR(10) PRIMARY KEY,
    이름 VARCHAR(20) NOT NULL,
    학과 VARCHAR(30)
);
```


학습 2 고급 SQL 작성하기

2-1. 테이블 외의 데이터 사전 조회

학습목표 조인과 서브쿼리를 활용하여 두 개 이상의 테이블에서 데이터를 조회할 수 있다.

가. 조인(JOIN)

- 1) INNER JOIN은 두 테이블에서 조건을 만족하는 행만 반환하고, OUTER JOIN(LEFT/RIGHT)은 한쪽 테이블의 모든 행을 포함하여 반환한다.

나. 서브쿼리(Subquery)

- 1) 서브쿼리는 SQL문 안에 포함된 또 다른 SQL문으로, 조건절이나 FROM절에서 다른 쿼리의 결과를 활용할 때 사용한다.

다. 집합 연산자

- 1) UNION(합집합), INTERSECT(교집합), MINUS/EXCEPT(차집합) 연산자로 여러 SELECT 결과를 조합할 수 있다.
- 2) 집합 연산자를 사용하려면 두 SELECT문의 칼럼 개수와 자료형이 일치해야 하며, UNION은 기본적으로 중복 행을 제거하고 UNION ALL은 중복을 포함하여 반환한다.

▶ 예시: JOIN 구문

```
SELECT A.학번, A.이름, B.과목코드  
FROM 학생 A INNER JOIN 수강 B ON A.학번 = B.학번;
```

2-2. 인덱스와 뷰 작성

학습목표 인덱스와 뷰를 생성하여 조회 성능을 높이고 데이터 접근을 단순화할 수 있다.

가. 인덱스(Index)

1) 인덱스는 특정 칼럼에 대한 검색 속도를 높이기 위한 데이터 구조로, 조회 성능은 향상되지만 삽입·수정·삭제 시 유지 비용이 발생한다.

나. 뷰(View)

1) 뷰는 하나 이상의 물리 테이블을 기반으로 만든 가상의 논리 테이블로, 복잡한 조인을 단순화하고 보안을 위해 특정 칼럼만 노출할 때 활용한다.

2) 뷰는 실제 데이터를 저장하지 않고 SELECT문 자체를 저장하므로, 원본 테이블의 데이터가 변경되면 뷰를 통해 조회되는 값도 함께 변경된다.

[표 8-2] 인덱스와 뷰 비교

구분	특징	장점
인덱스	칼럼 값 기반 검색 구조	조회 속도 향상
뷰	SELECT문 기반 가상 테이블	쿼리 단순화, 접근 제어

1. 데이터 사전과 DDL

[핵심 요약]

가. 데이터 사전에는 테이블·칼럼 등 메타데이터가 저장된다.

나. DDL은 CREATE, ALTER, DROP으로 구성된다.

[빈칸 채우기]

① 테이블 구조를 정의하는 언어를 ()이라 한다.

② 테이블을 삭제하는 DDL 명령어는 ()이다.

2. DML과 DCL

[핵심 요약]

가. DML은 데이터 조회·조작 명령어로 구성된다.

나. DCL은 권한과 트랜잭션을 제어한다.

[빈칸 채우기]

① 데이터를 조회하는 DML 명령어는 ()이다.

② 트랜잭션을 확정하는 DCL 명령어는 ()이다.

3. 조인·서브쿼리와 인덱스·뷰

[핵심 요약]

가. 조인은 두 개 이상의 테이블에서 데이터를 결합하여 조회한다.

나. 인덱스는 조회 성능을, 뷰는 쿼리 단순화를 돕는다.

[빈칸 채우기]

① SQL문 안에 포함된 또 다른 SQL문을 ()라 한다.

② 하나 이상의 테이블로 만든 가상의 논리 테이블을 ()라 한다.

[정리 문제 - 서술형]

1. DDL, DML, DCL의 차이를 명령어 예시와 함께 서술하시오.
2. INNER JOIN과 OUTER JOIN의 차이를 설명하시오.
3. 인덱스를 과도하게 생성할 경우 발생할 수 있는 단점을 서술하시오.
4. UNION과 UNION ALL의 차이를 설명하시오.
5. 뷰(View)를 사용하는 목적을 두 가지 이상 서술하시오.

[정답]

1. DDL은 CREATE·ALTER·DROP으로 객체 구조를 정의하고, DML은 SELECT·INSERT·UPDATE·DELETE로 데이터를 조작하며, DCL은 GRANT·REVOKE·COMMIT·ROLLBACK으로 권한과 트랜잭션을 제어한다.

2. INNER JOIN은 조건을 만족하는 행만 반환하고, OUTER JOIN은 한쪽 테이블의 모든 행을 포함하여 반환한다.
3. 인덱스가 많아지면 조회 속도는 향상되지만 삽입·수정·삭제 시 인덱스도 함께 갱신해야 하므로 쓰기 성능이 저하되고 저장 공간이 늘어난다.
4. UNION은 중복 행을 제거하여 합집합을 반환하고, UNION ALL은 중복을 제거하지 않고 모든 행을 그대로 반환한다.
5. 복잡한 조인문을 단순화하여 재사용할 수 있고, 특정 칼럼만 노출하여 민감한 정보에 대한 접근을 제한할 수 있다.

SQL 활용 - 형성평가

1. 테이블 구조를 정의하는 언어는?
① DML ② DDL ③ DCL ④ TCL
2. 데이터를 조회·조작하는 언어는?
① DDL ② DML ③ DCL ④ ERD
3. 트랜잭션을 취소하는 DCL 명령어는?
① COMMIT ② ROLLBACK ③ GRANT ④ REVOKE
4. 행을 유일하게 식별하며 NULL을 허용하지 않는 것은?
① 외래키 ② 기본키 ③ 인덱스 ④ 뷰
5. 다른 테이블의 기본키를 참조하는 것은?
① 외래키 ② 기본키 ③ 인덱스 ④ 트리거
6. 조건을 만족하는 행만 반환하는 조인은?
① OUTER JOIN ② INNER JOIN ③ CROSS JOIN ④ SELF JOIN
7. SQL문 안에 포함된 또 다른 SQL문은?
① 서브쿼리 ② 트리거 ③ 인덱스 ④ 뷰
8. 검색 속도 향상을 위한 데이터 구조는?
① 뷰 ② 인덱스 ③ 트랜잭션 ④ 서브쿼리
9. 하나 이상의 테이블로 만든 가상의 논리 테이블은?
① 뷰 ② 인덱스 ③ 스키마 ④ 도메인
10. 두 SELECT 결과의 합집합을 구하는 연산자는?
① UNION ② INTERSECT ③ MINUS ④ JOIN
11. 테이블을 삭제하는 DDL 명령어를 쓰시오.
12. 데이터를 삽입하는 DML 명령어를 쓰시오.
13. 권한을 부여하는 DCL 명령어를 쓰시오.
14. 칼럼 값의 중복을 허용하지 않는 제약 조건의 명칭을 쓰시오.
15. 뷰가 실제로 저장하는 것은 데이터가 아니라 무엇인지 쓰시오.

[정답 및 해설]

문항	정답	해설
1	②	DDL은 객체 구조를 정의한다.
2	②	DML은 데이터를 조회·조작한다.
3	②	ROLLBACK은 트랜잭션을 취소한다.
4	②	기본키는 NULL 불가, 행을 유일 식별한다.
5	①	외래키는 다른 테이블의 기본키를 참조한다.
6	②	INNER JOIN은 조건 만족 행만 반환한다.
7	①	서브쿼리는 SQL문 안의 SQL문이다.
8	②	인덱스는 검색 속도를 높인다.
9	①	뷰는 가상의 논리 테이블이다.
10	①	UNION은 합집합 연산자이다.
11	DROP	테이블을 삭제하는 DDL 명령어이다.
12	INSERT	데이터를 삽입하는 DML 명령어이다.
13	GRANT	권한을 부여하는 DCL 명령어이다.
14	UNIQUE	칼럼 값의 중복을 허용하지 않는 제약이다.
15	SELECT문(질의문)	뷰는 SELECT문 자체를 저장한다.

9. 프로그래밍 언어 응용 [LM2001020230_23v5]

학습 1 언어 특성 활용하기

1-1. 언어별 특성 파악

학습목표 프로그래밍 언어별 특성을 파악하고 목적에 맞는 언어를 설명할 수 있다.

가. 저급언어와 고급언어

- 1) 저급언어(기계어, 어셈블리어)는 컴퓨터가 직접 이해할 수 있어 실행 속도가 빠르지만 사람이 읽고 작성하기 어렵다.
- 2) 고급언어(C, Java, Python 등)는 사람이 이해하기 쉽도록 작성되어 가독성이 높으며, 컴파일러·인터프리터를 통해 기계어로 번역되어야 실행된다.

나. 번역 방식에 따른 언어 구분

- 1) 원시 코드(소스코드)는 개발자가 작성한 코드이며, 목적 코드(실행 파일)는 번역을 거쳐 컴퓨터가 실행할 수 있는 형태로 변환된 코드이다.
- 2) 바이트코드 언어는 고급언어를 중간 언어로 컴파일한 뒤 가상머신에서 실행되어 플랫폼 독립성을 확보한다.

[표 9-1] 언어 유형별 특징

구분	특징	대표 언어
저급언어	실행 속도 빠름, 이식성 낮음	기계어, 어셈블리어
고급언어(컴파일)	가독성 높음, 실행 전 전체 번역	C, C++
고급언어(인터프리터)	한 줄씩 해석 실행, 이식성 높음	Python, JavaScript

다. 언어 선택 기준

- 1) 실행 속도가 중요한 시스템 프로그래밍에는 컴파일 언어를, 빠른 개발과 유연성이 중요한 업무에는 인터프리터 언어를 선택하는 경향이 있다.

1-2. 애플리케이션 구현 및 최적화

학습목표 프로그래밍 언어의 특성을 활용하여 애플리케이션을 구현하고 최적화할 수 있다.

가. 코드 리뷰

- 1) 코드 리뷰는 동료 개발자가 소스코드를 검토하여 잠재적 결함과 개선점을 사전에 찾아내는 활동이다.
- 2) 코드 리뷰는 결함을 조기에 발견할 뿐 아니라, 팀원 간 코딩 스타일과 설계 지식을 공유하여 전반적인 코드 품질을 상향평준화하는 효과가 있다.

나. 리팩토링(Refactoring)

- 1) 리팩토링은 외부 동작은 그대로 유지하면서 내부 코드 구조를 개선하는 작업으로, 중복 코드 제거, 메서드 분리, 이름 개선 등이 대표적인 대상이다.
- 2) 리팩토링 전후로 동일한 테스트 코드를 실행하여 결과가 같은지 확인하면, 구조를 개선하면서도 기능이 깨지지 않았음을 보장할 수 있다.

다. 성능 최적화

- 1) 불필요한 반복 연산 제거, 자료구조 선택 최적화, 캐싱 활용 등을 통해 실행 속도와 메모리 사용을 개선한다.
- 2) 최적화는 반드시 성능 측정(프로파일링)을 통해 병목 구간을 먼저 파악한 뒤 진행해야 하며, 근거 없이 코드를 수정하면 가독성만 떨어뜨릴 수 있다.

▶ 실무 Tip

성급한 최적화는 만약의 근원이라는 격언처럼, 실제로 느린 부분을 측정으로 확인한 뒤 그 부분만 집중적으로 개선하는 것이 효율적이다.

학습 2 라이브러리 활용하기

2-1. 라이브러리 선정

학습목표 개발 목적에 적합한 라이브러리를 조사하고 선정할 수 있다.

가. 모듈·패키지·라이브러리의 개념

1) 모듈은 특정 기능을 수행하는 코드 단위, 패키지는 모듈을 묶은 집합, 라이브러리는 재사용 가능한 코드·패키지의 모음으로 프로그램 개발에 필요한 기능을 제공한다.

나. 라이브러리 선정 기준

- 1) 기능 적합성, 라이선스 조건, 유지보수 활성화도(커뮤니티 활동), 문서화 수준, 성능을 종합적으로 고려하여 선정한다.
- 2) 오픈소스 라이브러리를 사용할 때는 라이선스 유형(MIT, Apache 2.0, GPL 등)에 따라 상업적 활용·수정·배포 조건이 다르므로 반드시 사전에 확인해야 한다.

[표 9-1a] 대표적인 오픈소스 라이선스 비교

라이선스	특징
MIT	매우 관대함, 저작권 표시만 하면 자유롭게 사용·수정·배포 가능
Apache 2.0	MIT와 유사하나 특허 관련 조항이 명시됨
GPL	이 라이브러리를 사용한 소스코드도 동일 라이선스로 공개해야 함(카피레프트)

2-2. 라이브러리 구성 및 적용

학습목표 *선택한 라이브러리를 프로젝트에 구성하고 적용할 수 있다.*

가. 의존성 관리

1) Maven·Gradle(Java), pip(Python), npm(JavaScript)과 같은 패키지 관리 도구를 이용하여 라이브러리의 버전과 의존 관계를 관리한다.

나. 라이브러리 적용과 검증

1) 라이브러리를 프로젝트에 추가한 뒤 예제 코드로 정상 동작을 검증하고, 버전 충돌이 없는지 확인한다.

2) 여러 라이브러리가 동일한 하위 의존성을 서로 다른 버전으로 요구하는 의존성 충돌이 발생할 수 있으므로, 의존성 트리를 점검하여 버전을 통일한다.

[표 9-2] 언어별 패키지 관리 도구

언어	패키지(의존성) 관리 도구
Java	Maven, Gradle
Python	pip, conda
JavaScript	npm, yarn

1. 저급언어와 고급언어

[핵심 요약]

- 가. 저급언어는 실행 속도가 빠르지만 가독성이 낮다.
- 나. 고급언어는 사람이 이해하기 쉽고 번역 과정을 거쳐 실행된다.

[빈칸 채우기]

- ① 컴퓨터가 직접 이해할 수 있는 언어를 ()라 한다.
- ② 사람이 이해하기 쉽게 작성한 언어를 ()라 한다.

2. 리팩토링과 최적화

[핵심 요약]

- 가. 리팩토링은 외부 동작을 유지하며 내부 구조를 개선하는 작업이다.
- 나. 코드 리뷰는 동료가 코드를 검토하여 결함을 사전에 찾는 활동이다.

[빈칸 채우기]

- ① 외부 동작을 유지하며 코드 구조를 개선하는 작업을 ()이라 한다.
- ② 동료 개발자가 소스코드를 검토하는 활동을 코드 ()라 한다.

3. 라이브러리와 의존성 관리

[핵심 요약]

- 가. 라이브러리는 재사용 가능한 코드·패키지의 모음이다.
- 나. 패키지 관리 도구로 라이브러리 버전과 의존 관계를 관리한다.
- 다. 오픈소스 라이브러리는 라이선스 조건을 반드시 확인해야 한다.

[빈칸 채우기]

- ① Java 진영의 대표적인 패키지 관리 도구는 ()이다.
- ② Python 진영의 대표적인 패키지 관리 도구는 ()이다.
- ③ 저작권 표시만 하면 자유롭게 사용·수정·배포할 수 있는 관대한 오픈소스 라이선스는 ()이다.

[정리 문제 - 서술형]

- 1. 저급언어와 고급언어의 차이를 실행 속도와 가독성 측면에서 설명하시오.
- 2. 리팩토링이 필요한 이유를 서술하시오.
- 3. 라이브러리를 선정할 때 고려해야 할 기준을 두 가지 이상 서술하시오.
- 4. 성능 최적화를 진행하기 전에 프로파일링(성능 측정)을 먼저 해야 하는 이유를 서술하시오.
- 5. GPL 라이선스를 가진 라이브러리를 사용할 때 주의해야 할 점을 서술하시오.

[정답]

1. 저급언어는 실행 속도가 빠르지만 가독성이 낮고, 고급언어는 가독성이 높지만 번역 과정을 거쳐야 실행되어 상대적으로 실행 속도가 느릴 수 있다.
2. 코드가 반복적으로 수정되며 구조가 복잡해지면 유지보수가 어려워지므로, 외부 동작은 유지한 채 내부 구조를 개선하여 가독성과 유지보수성을 높이기 위함이다.
3. 기능 적합성, 라이선스 조건, 커뮤니티의 유지보수 활성화도, 문서화 수준, 성능 등을 고려해야 한다.
4. 실제 병목 구간을 측정 없이 추측만으로 수정하면 효과가 없거나 오히려 가독성만 떨어뜨릴 수 있으므로, 측정을 통해 실제로 느린 부분을 확인한 뒤 그 부분에 집중해야 한다.
5. GPL은 카피레프트 라이선스로, 이를 사용한 소스코드도 동일한 라이선스로 공개해야 할 수 있으므로 상업적 프로젝트에 적용할 때는 사전에 법적 검토가 필요하다.

프로그래밍 언어 응용 - 형성평가

1. 컴퓨터가 직접 이해할 수 있는 언어는?
① 고급언어 ② 저급언어 ③ 스크립트 언어 ④ 마크업 언어
2. 사람이 이해하기 쉽게 작성되어 번역 과정이 필요한 언어는?
① 저급언어 ② 고급언어 ③ 기계어 ④ 어셈블리어
3. 개발자가 작성한 코드를 무엇이라 하는가?
① 목적 코드 ② 원시 코드 ③ 실행 파일 ④ 바이트 코드
4. 외부 동작은 유지하고 내부 구조만 개선하는 작업은?
① 디버깅 ② 리팩토링 ③ 컴파일 ④ 배포
5. 동료 개발자가 소스코드를 검토하는 활동은?
① 코드 리뷰 ② 코드 배포 ③ 코드 삭제 ④ 코드 복사
6. 재사용 가능한 코드·패키지의 모음은?
① 라이브러리 ② 데이터베이스 ③ 네트워크 ④ 운영체제
7. Java 진영의 대표적인 패키지 관리 도구는?
① npm ② pip ③ Maven ④ yarn
8. Python 진영의 대표적인 패키지 관리 도구는?
① pip ② Maven ③ Gradle ④ npm
9. 고급언어를 중간 언어로 컴파일 후 가상머신에서 실행하는 방식은?
① 기계어 실행 ② 바이트코드 실행 ③ 어셈블리 실행 ④ 하드코딩
10. 라이브러리 선정 기준으로 가장 거리가 먼 것은?
① 기능 적합성 ② 라이선스 조건 ③ 개발자의 개인 취향 ④ 유지보수 활성화도
11. 모듈을 묶어 놓은 집합을 무엇이라 하는가?
12. 불필요한 반복 연산 제거 등을 통해 실행 속도를 개선하는 활동을 쓰시오.
13. JavaScript 진영의 대표적인 패키지 관리 도구를 쓰시오.

14. 성능 최적화 전에 병목 구간을 파악하기 위해 수행하는 활동을 쓰시오.

15. 사용한 소스코드도 동일 라이선스로 공개해야 하는 오픈소스 라이선스 유형을 쓰시오.

[정답 및 해설]

문항	정답	해설
1	②	저급언어는 컴퓨터가 직접 이해한다.
2	②	고급언어는 번역 과정을 거쳐 실행된다.
3	②	개발자가 작성한 코드는 원시 코드이다.
4	②	리팩토링은 내부 구조 개선 작업이다.
5	①	코드 리뷰는 동료의 코드 검토 활동이다.
6	①	라이브러리는 재사용 가능한 코드 모음이다.
7	③	Maven은 Java 패키지 관리 도구이다.
8	①	pip은 Python 패키지 관리 도구이다.
9	②	바이트코드 실행은 가상머신에서 이뤄진다.
10	③	개인 취향은 선정 기준으로 부적절하다.
11	패키지	모듈을 묶어 놓은 집합이다.
12	성능 최적화	실행 속도·메모리 사용을 개선하는 활동이다.
13	npm(또는 yarn)	JavaScript 패키지 관리 도구이다.
14	프로파일링(성능 측정)	병목 구간을 파악하기 위한 측정 활동이다.
15	GPL	카피레프트 방식의 오픈소스 라이선스이다.

II. 실전 모의고사

1. 1회

※ 1~18번은 객관식, 19~25번은 단답형·서술형 문항입니다. 문항을 읽고 답하시오.

1. 애플리케이션 배포 절차 중 가장 먼저 수행하는 것은?
① 빌드 ② 배포 환경 구성 ③ 소스 검증 ④ 운영 배포
2. 소스코드를 실행하지 않고 분석하는 정적 검증 기법은?
① 코드 인스펙션 ② 부하 테스트 ③ 인수 테스트 ④ 회귀 테스트
3. V-모델에서 시스템 테스트와 대응하는 개발 단계는?
① 요구사항 분석 ② 설계 ③ 구현 ④ 형상관리
4. 결함 관리 프로세스에서 결함을 등록하는 단계는?
① 결함 계획 ② 결함 기록 ③ 결함 수정 ④ 결함 종료
5. OSI 7계층 중 전송 계층에 해당하는 프로토콜은?
① HTTP ② TCP ③ ARP ④ Ethernet
6. 운영체제와 응용소프트웨어 사이에서 공통 기능을 제공하는 것은?
① 컴파일러 ② 미들웨어 ③ 텍스트 에디터 ④ 브라우저
7. 데이터 중복과 이상 현상을 줄이기 위한 과정은?
① 캡슐화 ② 정규화 ③ 다중화 ④ 가상화
8. 오픈소스이며 무료로 배포 가능한 운영체제는?
① Windows ② UNIX ③ Linux ④ iOS
9. 파일 접근 권한을 변경하는 명령어는?
① ls ② chmod ③ cd ④ mkdir

10. 사용자가 생각을 소리 내어 말하며 과업을 수행하는 사용성 테스트 기법은?

- ① 휴리스틱 평가 ② 씹크 얼라우드 ③ A/B 테스트 ④ 코드 인스펙션

11. UI 이슈의 심각도 분류에 해당하지 않는 것은?

- ① 치명적 ② 중대 ③ 경미 ④ 영구적

12. 상위 클래스의 속성·메서드를 하위 클래스가 물려받는 특성은?

- ① 캡슐화 ② 상속 ③ 다형성 ④ 추상화

13. 별도 컴파일 없이 한 줄씩 해석·실행되는 언어는?

- ① 컴파일 언어 ② 스크립트 언어 ③ 기계어 ④ 어셈블리어

14. 화면의 구조와 콘텐츠를 정의하는 언어는?

- ① CSS ② HTML ③ SQL ④ XML

15. 여러 브라우저에서 동일하게 동작하는지 확인하는 것은?

- ① 웹 접근성 ② 웹 호환성 ③ 웹 보안 ④ 웹 성능

16. 테이블 구조를 정의하는 SQL 언어는?

- ① DML ② DDL ③ DCL ④ TCL

17. 조건을 만족하는 행만 반환하는 조인은?

- ① OUTER JOIN ② INNER JOIN ③ CROSS JOIN ④ SELF JOIN

18. 외부 동작은 유지하며 내부 코드 구조를 개선하는 작업은?

- ① 디버깅 ② 리팩토링 ③ 컴파일 ④ 배포

19. 소스 반출부터 빌드·테스트·패키징까지 자동화된 배포 흐름 전체를 무엇이라 하는가?

▶ 정답: -----

20. 결함이 시스템에 미치는 영향의 크기가 아니라, 처리해야 하는 시급성을 나타내는 개념은?

▶ 정답: -----

21. 정규화가 데이터베이스 설계에서 필요한 이유를 이상 현상과 관련지어 서술하시오.

▶ 서술형 - 아래 여백에 답을 작성하시오.

22. 사용자가 콘텐츠 카드를 스스로 분류하게 하여 메뉴 구조를 검증하는 사용성 테스트 기법은?

▶ 정답: -----

23. 객체지향에서 상위 클래스의 속성과 메서드를 하위 클래스가 물려받는 특성은?

▶ 정답: -----

24. 클라이언트 측 유효성 검사만으로 충분하지 않은 이유를 서술하시오.

▶ 서술형 - 아래 여백에 답을 작성하시오.

25. 재사용 가능한 코드·패키지의 모음으로, npm·pip과 같은 도구로 관리되는 것은?

▶ 정답: -----

2. 2회

※ 1~18번은 객관식, 19~25번은 단답형·서술형 문항입니다. 문항을 읽고 답하시오.

1. 장애 발생 시 이전 정상 버전으로 되돌리는 작업은?
① 체크인 ② 롤백 ③ 캡슐화 ④ 정규화
2. 테스트 관리 도구가 제공하는 기능으로 옳은 것은?
① 결함 추적·기록 ② 서버 가상화 ③ 이미지 편집 ④ 문서 번역
3. 최종 사용자가 운영 여부를 결정하기 위해 수행하는 테스트는?
① 단위 테스트 ② 통합 테스트 ③ 인수 테스트 ④ 회귀 테스트
4. 결함이 시스템에 미치는 영향의 크기를 나타내는 개념은?
① 우선순위 ② 심각도 ③ 커버리지 ④ 재현율
5. 사설 IP 대역에 해당하는 것은?
① 8.8.8.8 ② 192.168.0.0/16 ③ 1.1.1.1 ④ 203.0.113.0
6. 시스템 간 비동기 통신을 지원하는 미들웨어는?
① 메시지 큐 ② 컴파일러 ③ 텍스트 에디터 ④ 뷰어
7. 외래키가 참조 테이블에 존재해야 한다는 제약은?
① 개체 무결성 ② 참조 무결성 ③ 도메인 무결성 ④ 키 무결성
8. 코드 편집·빌드·디버깅을 통합 제공하는 도구는?
① IDE ② 브라우저 ③ 스프레드시트 ④ 메일 클라이언트
9. 분산 형상관리 도구의 대표적인 예는?
① Git ② Excel ③ Photoshop ④ Notepad
10. 실제 시스템 사용자에게 가까운 속성의 참여자를 모집해야 하는 이유는?
① 비용 절감 ② 테스트 결과의 타당성 확보 ③ 일정 단축 ④ 서버 부하 감소

11. UI 개선 결과보고서에 포함되어야 할 항목이 아닌 것은?

- ① 테스트 개요 ② 주요 이슈 ③ 향후 계획 ④ 서버 IP 대역

12. 동일한 이름의 메서드가 객체마다 다르게 동작하는 특성은?

- ① 캡슐화 ② 상속 ③ 다형성 ④ 정규화

13. 클라이언트에서 실행되어 동적 UI를 구현하는 대표 언어는?

- ① SQL ② JavaScript ③ C언어 ④ 어셈블리어

14. 필수 입력 여부와 형식을 검증하는 작업은?

- ① 유효성 검사 ② 정적 테스트 ③ 회귀 테스트 ④ 부하 테스트

15. HTML/CSS 문법 오류를 검사하는 도구는?

- ① Git ② W3C Markup Validator ③ Maven ④ JUnit

16. 데이터를 조회하는 DML 명령어는?

- ① CREATE ② SELECT ③ GRANT ④ DROP

17. 검색 속도 향상을 위한 데이터 구조는?

- ① 뷰 ② 인덱스 ③ 트랜잭션 ④ 서브쿼리

18. 사람이 이해하기 쉽게 작성되어 번역 과정이 필요한 언어는?

- ① 저급언어 ② 고급언어 ③ 기계어 ④ 어셈블리어

19. 결함이 재현되지 않을 때 부여하는 결함 상태는?

▶ 정답: -----

20. 하나 이상의 물리 테이블을 기반으로 만든 가상의 논리 테이블은?

▶ 정답: -----

21. 코드 인스펙션과 동적 테스트의 차이를 실행 여부를 기준으로 서술하시오.

▶ 서술형 - 아래 여백에 답을 작성하시오.

22. 여러 브라우저에서 화면이 동일하게 동작하는지 확인하는 품질 항목은?

▶ 정답: -----

23. 외부 동작은 유지한 채 내부 코드 구조만 개선하는 작업은?

▶ 정답: -----

24. 인덱스를 과도하게 생성했을 때 발생할 수 있는 단점을 서술하시오.

▶ 서술형 - 아래 여백에 답을 작성하시오.

25. 시스템 간 비동기 통신을 지원하여 장애 전파를 완화하는 미들웨어는?

▶ 정답: -----

3. 3회

※ 1~18번은 객관식, 19~25번은 단답형·서술형 문항입니다. 문항을 읽고 답하십시오.

1. Java 웹 애플리케이션 배포에 주로 쓰이는 패키지 형식은?

- ① exe ② war ③ zip ④ txt

2. 구조 기반(화이트박스) 테스트의 관점은?

- ① 요구사항명세서 기반 ② 프로그램 내부 구조·복잡도 ③ 사용자 인터뷰 ④ 서버 부하

3. 수정 사항이 다른 기능에 영향을 주지 않는지 확인하는 테스트는?

- ① 회귀 테스트 ② 인수 테스트 ③ 단위 테스트 ④ 통합 테스트

4. 신뢰성보다 속도를 중시하는 전송 프로토콜은?

- ① TCP ② UDP ③ FTP ④ SSH

5. WAS의 주된 역할로 옳은 것은?

- ① 정적 파일 저장 ② 애플리케이션 실행 및 배포 관리 ③ 이메일 발송 ④ 방화벽 설정

6. 데이터베이스 구조를 개체와 관계로 표현하는 다이어그램은?

- ① UML ② ERD ③ DFD ④ Flowchart

7. 리눅스에서 실행 중인 프로세스 상태를 확인하는 명령어는?

- ① ps ② rm ③ mv ④ mkdir

8. 전문가가 이론과 경험으로 사용성을 평가하는 기법은?

- ① 썬크 얼라우드 ② 휴리스틱 평가 ③ A/B 테스트 ④ 코드 인스펙션

9. 이슈 개선 우선순위를 정할 때 가장 중요하게 고려할 것은?

- ① 이슈의 심각도와 영향 ② 개발자의 선호 ③ 코드 라인 수 ④ 서버 위치

10. 데이터와 메서드를 하나로 묶어 내부 구현을 숨기는 특성은?

- ① 상속 ② 다형성 ③ 캡슐화 ④ 추상화

11. 접근 제어자에 해당하지 않는 것은?

- ① public ② private ③ protected ④ dynamic

12. 다양한 화면 크기에 대응하는 레이아웃 기법은?

- ① 고정형 레이아웃 ② 반응형 레이아웃 ③ 정적 레이아웃 ④ 단일형 레이아웃

13. 신체적 제약이 있는 사용자도 콘텐츠를 이용할 수 있도록 하는 원칙은?

- ① 웹 표준 ② 웹 접근성 ③ 웹 호환성 ④ 웹 보안

14. 트랜잭션을 취소하는 DCL 명령어는?

- ① COMMIT ② ROLLBACK ③ GRANT ④ REVOKE

15. 하나 이상의 테이블로 만든 가상의 논리 테이블은?

- ① 뷰 ② 인덱스 ③ 스키마 ④ 도메인

16. 동료 개발자가 소스코드를 검토하는 활동은?

- ① 코드 리뷰 ② 코드 배포 ③ 코드 삭제 ④ 코드 복사

17. 재사용 가능한 코드·패키지의 모음은?

- ① 라이브러리 ② 데이터베이스 ③ 네트워크 ④ 운영체제

18. Python 진영의 대표적인 패키지 관리 도구는?

- ① pip ② Maven ③ Gradle ④ npm

19. 배포 후 장애가 발생했을 때 이전 정상 버전으로 되돌리는 작업은?

▶ 정답: -----

20. 데이터베이스 구조를 개체와 관계로 표현하는 다이어그램은?

▶ 정답: -----

21. 휴리스틱 평가의 장점과 단점을 각각 한 가지씩 서술하시오.

▶ 서술형 - 아래 여백에 답을 작성하시오.

22. HTML에서 이미지에 대체 텍스트를 지정하는 속성은?

▶ 정답: -----

23. 동일한 이름의 메서드가 객체마다 다르게 동작하는 객체지향 특성은?

▶ 정답: -----

24. 저급언어와 고급언어의 차이를 실행 속도와 가독성 측면에서 서술하시오.

▶ 서술형 - 아래 여백에 답을 작성하시오.

25. Python 진영의 대표적인 패키지 관리 도구는?

▶ 정답: -----

4. 4회

※ 1~18번은 객관식, 19~25번은 단답형·서술형 문항입니다. 문항을 읽고 답하십시오.

1. 배포 단위로 Java 웹 애플리케이션에 주로 쓰이지 않는 패키지 형식은?
① war ② jar ③ ear ④ doc
2. 명세 기반(블랙박스) 테스트의 대표 기법은?
① 문장 커버리지 ② 경계값 분석 ③ 분기 커버리지 ④ 조건 커버리지
3. 결함 상태 중 재현되지 않을 때 부여하는 상태는?
① Open ② Fixed ③ Reject ④ Closed
4. 캡슐화, 다중화, 오류제어를 정의하는 통신 요소는?
① 미들웨어 ② 프로토콜 ③ 인덱스 ④ 뷰
5. IT 자원 관리형 미들웨어에 해당하는 것은?
① 방화벽 ② WAS ③ API Gateway ④ VPN
6. 3정규형까지의 정규화가 방지하는 것은?
① 캡슐화 ② 이상 현상 ③ 다형성 ④ 오버로딩
7. 운영체제의 기능이 아닌 것은?
① 파일 관리 ② CPU 스케줄링 ③ 웹 디자인 ④ 사용자 인터페이스 제공
8. Eclipse, IntelliJ와 같은 도구를 통칭하는 용어는?
① 컴파일러 ② IDE ③ 인터프리터 ④ 링커
9. 참여자가 과업을 수행하며 생각을 말로 표현하는 기법은?
① 카드 소팅 ② 씽크 얼라우드 ③ A/B 테스트 ④ 코드 리뷰
10. UI 결과보고서에 포함되지 않는 항목은?
① 개선 전후 비교 ② 향후 계획 ③ 주요 이슈 ④ 데이터베이스 인덱스 목록

11. 하향식 설계의 특징으로 옳은 것은?

- ① 하위에서 상위로 통합 ② 상위에서 하위로 세분화 ③ 무작위 설계 ④ 테스트 우선 설계

12. 인터페이스의 역할로 옳은 것은?

- ① 메서드 시그니처만 정의해 규약 강제 ② 데이터 저장 ③ 화면 렌더링 ④ 네트워크 전송

13. Python의 동적 타이핑 특징으로 옳은 것은?

- ① 자료형을 실행 시점에 결정 ② 컴파일 시 자료형 고정 ③ 자료형이 없음 ④ 항상 정수형만 사용

14. 반응형 웹 구현에 사용되는 CSS 기법은?

- ① 미디어 쿼리 ② 서브쿼리 ③ 트리거 ④ 인덱스

15. 웹 접근성 준수를 위해 이미지에 지정하는 속성은?

- ① src ② alt ③ class ④ id

16. 서브쿼리를 사용하는 이유로 옳은 것은?

- ① 다른 쿼리의 결과를 조건이나 FROM절에서 활용 ② 인덱스를 삭제하기 위해 ③ 트랜잭션을 취소하기 위해 ④ 권한을 부여하기 위해

17. GPL 라이선스의 특징은?

- ① 완전 상업적 자유 ② 카피레프트(동일 라이선스 공개 의무) ③ 특허 조항만 존재 ④ 라이선스 없음

18. 코드 리뷰의 효과로 가장 거리가 먼 것은?

- ① 결함 조기 발견 ② 코딩 스타일 공유 ③ 실행 속도 자동 향상 ④ 팀 지식 공유

19. 실행 전 소스 전체를 기계어로 번역한 뒤 실행하는 방식의 언어를 무엇이라 하는가?

▶ 정답: -----

20. 프로토콜의 3요소는 구문, 의미와 그리고 무엇인가?

▶ 정답: -----

21. 배포 파이프라인에서 각 단계가 이전 단계의 성공 여부에 따라서만 진행되도록 구성하는 이유를 서술하시오.

▶ 서술형 - 아래 여백에 답을 작성하시오.

22. 테스트가 소스코드를 얼마나 실행했는지를 나타내는 지표는?

▶ 정답: -----

23. 여러 SELECT 결과에서 중복을 제거하지 않고 모두 반환하는 집합 연산자는?

▶ 정답: -----

24. 사용성 테스트 진행자가 유도 질문을 피해야 하는 이유를 서술하시오.

▶ 서술형 - 아래 여백에 답을 작성하시오.

25. 개발 환경 구축 시 접속 계정과 권한을 최소한으로 부여하는 보안 원칙은?

▶ 정답: -----

5. 정답 및 해설

[1회 정답 및 해설]

문항	정답	해설
1	②	환경 구성이 가장 먼저 이루어진다.
2	①	코드 인스펙션은 실행 없이 소스를 분석한다.
3	②	설계 단계는 시스템 테스트와 대응한다.
4	②	결함 기록 단계에서 결함 정보를 DB에 등록한다.
5	②	전송 계층은 TCP/UDP를 사용한다.
6	②	미들웨어는 OS와 응용SW 사이의 공통 기능을 제공한다.
7	②	정규화는 데이터 중복 최소화를 목적으로 한다.
8	③	Linux는 오픈소스 기반으로 무료 배포가 가능하다.
9	②	chmod는 접근 권한을 변경한다.
10	②	썬크 얼라우드는 생각을 말하며 과업을 수행하는 기법이다.
11	④	영구적은 심각도 분류 기준이 아니다.
12	②	상속은 속성·메서드를 물려받는 특성이다.
13	②	스크립트 언어는 인터프리터가 해석·실행한다.
14	②	HTML은 구조와 콘텐츠를 정의한다.
15	②	웹 호환성은 여러 브라우저에서의 동일 동작을 확인한다.
16	②	DDL은 테이블 등 객체 구조를 정의한다.

17	②	INNER JOIN은 조건 만족 행만 반환한다.
18	②	리팩토링은 내부 구조 개선 작업이다.
19	(단답형) 배포 파이프라인	커밋부터 운영 반영까지의 전체 자동화 흐름을 배포 파이프라인이라 한다.
20	(단답형) 우선순위	심각도는 영향의 크기, 우선순위는 처리의 시급성을 의미한다.
21	(서술형 예시답안) 데이터 중복을 최소화하여 삽입·갱신·삭제 시 발생할 수 있는 이상 현상을 방지하기 위해 필요하다.	정규화의 목적은 중복 최소화를 통한 이상 현상 방지이다.
22	(단답형) 카드 소팅(Card Sorting)	카드 소팅은 정보 구조가 사용자의 인식과 일치하는지 검증하는 기법이다.
23	(단답형) 상속(Inheritance)	상속은 객체지향의 4대 특성 중 하나이다.
24	(서술형 예시답안) 클라이언트 측 검증은 개발자 도구 등으로 우회될 수 있으므로, 서버 측에서도 동일한 검증을 반드시 재수행해야 데이터 무결성을 보장할 수 있다.	클라이언트 검증은 우회 가능하므로 서버 측 검증이 병행되어야 한다.
25	(단답형) 라이브러리	라이브러리는 패키지 관리 도구를 통해 버전과 의존성을 관리한다.

[2회 정답 및 해설]

문항	정답	해설
1	②	롤백은 이전 버전으로 되돌리는 작업이다.
2	①	테스트 관리 도구는 결함 추적·기록 기능을 제공한다.
3	③	인수 테스트는 운영 여부 결정을 위한 테스트이다.
4	②	심각도는 결함이 미치는 영향의 크기이다.
5	②	192.168.0.0/16은 사설 IP 대역이다.
6	①	메시지 큐는 비동기 통신을 지원한다.
7	②	참조 무결성은 외래키의 유효성을 보장한다.
8	①	IDE는 개발 전 과정을 통합 지원한다.
9	①	Git은 대표적인 분산 형상관리 도구이다.
10	②	실제 사용자와 유사할수록 결과의 타당성이 높아진다.
11	④	서버 IP는 보고서 항목이 아니다.
12	③	다형성은 동일 메서드의 다른 동작을 의미한다.
13	②	JavaScript는 브라우저에서 동적 UI를 구현한다.
14	①	유효성 검사는 입력 형식을 검증한다.
15	②	W3C Markup Validator는 문법 오류를 검사한다.
16	②	SELECT는 데이터를 조회하는 DML 명령어이다.

17	②	인덱스는 검색 속도를 높인다.
18	②	고급언어는 번역 과정을 거쳐 실행된다.
19	(단답형) 반려(Reject)	재현되지 않는 결함은 반려 상태로 처리한다.
20	(단답형) 뷰(View)	뷰는 SELECT문 자체를 저장하는 가상 테이블이다.
21	(서술형 예시답안) 코드 인스펙션은 프로그램을 실행하지 않고 소스를 분석하는 정적 기법이고, 동적 테스트는 프로그램을 실제로 실행하여 동작을 검증하는 기법이다.	정적 기법은 미실행, 동적 기법은 실행을 전제로 한다.
22	(단답형) 웹 호환성	웹 호환성은 크로스 브라우징 검사를 통해 확인한다.
23	(단답형) 리팩토링(Refactoring)	리팩토링은 기능은 그대로 두고 구조만 개선하는 작업이다.
24	(서술형 예시답안) 삽입·수정·삭제 시 인덱스도 함께 갱신해야 하므로 쓰기 성능이 저하되고, 저장 공간이 추가로 소모된다.	인덱스는 조회 성능은 높이지만 쓰기 성능과 저장 공간에 부담을 준다.
25	(단답형) 메시지 큐(MQ)	메시지 큐는 생산자와 소비자 사이의 비동기 통신을 지원한다.

[3회 정답 및 해설]

문항	정답	해설
1	②	Java 웹 애플리케이션은 war로 배포되는 경우가 많다.
2	②	구조 기반은 내부 구조·복잡도를 검증한다.
3	①	회귀 테스트는 부작용 여부를 확인한다.
4	②	UDP는 비연결형으로 속도가 빠르다.
5	②	WAS는 애플리케이션 실행·배포를 관리한다.
6	②	ERD는 개체-관계 다이어그램이다.
7	①	ps는 프로세스 상태를 확인한다.
8	②	휴리스틱 평가는 전문가 판단 기반 기법이다.
9	①	심각도와 영향이 우선순위 결정 기준이다.
10	③	캡슐화는 데이터·메서드를 묶어 은닉한다.
11	④	dynamic은 접근 제어자가 아니다.
12	②	반응형 레이아웃은 다양한 화면 크기에 대응한다.
13	②	웹 접근성은 신체적 제약 사용자를 고려한다.
14	②	ROLLBACK은 트랜잭션을 취소한다.
15	①	뷰는 가상의 논리 테이블이다.
16	①	코드 리뷰는 동료의 코드 검토 활동이다.

17	①	라이브러리는 재사용 가능한 코드 모음이다.
18	①	pip은 Python 패키지 관리 도구이다.
19	(단답형) 롤백(Rollback)	롤백은 장애 시 이전 버전으로 되돌리는 절차이다.
20	(단답형) ERD(개체-관계 다이어그램)	ERD는 데이터베이스 설계 단계에서 구조를 표현하는 산출물이다.
21	(서술형 예시답안) 장점은 적은 비용으로 빠르게 문제를 발견할 수 있다는 것이고, 단점은 계량적 자료 확보가 어렵고 전문가와 실제 사용자의 시각 차이가 있을 수 있다는 것이다.	휴리스틱 평가는 신속하지만 정량화와 사용자 관점 반영에 한계가 있다.
22	(단답형) alt	alt 속성은 웹 접근성 준수를 위해 사용된다.
23	(단답형) 다형성(Polymorphism)	다형성은 객체지향의 4대 특성 중 하나이다.
24	(서술형 예시답안) 저급언어는 실행 속도가 빠르지만 가독성이 낮고, 고급언어는 가독성이 높지만 번역 과정을 거쳐야 하여 상대적으로 실행 속도가 느릴 수 있다.	저급언어와 고급언어는 실행 속도와 가독성 면에서 상반된 특성을 가진다.
25	(단답형) pip	pip은 Python 라이브러리의 설치·관리 도구이다.

[4회 정답 및 해설]

문항	정답	해설
1	④	doc은 배포 패키지 형식이 아니다.
2	②	경짓값 분석은 명세 기반 기법이다.
3	③	재현 불가 결함은 반려(Reject) 처리한다.
4	②	프로토콜이 통신 기능을 정의한다.
5	②	WAS는 IT 자원 관리형 미들웨어이다.
6	②	정규화는 이상 현상을 방지한다.
7	③	웹 디자인은 운영체제 기능이 아니다.
8	②	IDE는 통합개발환경이다.
9	②	씽크 얼라우드는 생각을 말하며 과업 수행하는 기법이다.
10	④	인덱스 목록은 UI 보고서 항목이 아니다.
11	②	하향식 설계는 상위→하위 세분화이다.
12	①	인터페이스는 메서드 시그니처로 규약을 강제한다.
13	①	동적 타이핑은 실행 시점에 자료형을 결정한다.
14	①	미디어 쿼리로 반응형 레이아웃을 구현한다.
15	②	alt 속성은 대체 텍스트를 지정한다.
16	①	서브쿼리는 다른 쿼리 결과를 활용한다.

17	②	GPL은 카피레프트 방식이다.
18	③	코드 리뷰가 실행 속도를 자동으로 높이지는 않는다.
19	(단답형) 컴파일 언어	컴파일 언어는 실행 전 전체를 번역하여 실행 속도가 빠르다.
20	(단답형) 타이밍(Timing)	프로토콜의 3요소는 구문·의미·타이밍이다.
21	(서술형 예시답안) 결함이 있는 코드나 검증되지 않은 변경 사항이 다음 단계, 특히 운영 환경까지 전달되는 것을 방지하기 위함이다.	단계별 성공 조건을 두는 것은 결함의 조기 차단을 위한 장치이다.
22	(단답형) 커버리지(Coverage)	구문·분기 커버리지 등이 대표적인 지표이다.
23	(단답형) UNION ALL	UNION은 중복을 제거하지만 UNION ALL은 모든 행을 그대로 반환한다.
24	(서술형 예시답안) 유도 질문은 참여자의 자연스러운 행동과 판단을 왜곡시켜, 실제 사용성 문제를 정확히 관찰하기 어렵게 만들기 때문이다.	진행자의 중립성은 관찰 데이터의 신뢰도를 좌우한다.
25	(단답형) 최소 권한 원칙	최소 권한 원칙은 보안 사고 가능성을 낮추기 위해 적용한다.